GATE

Be sure with Vidyalankar Since 1960

Pearl Centre, S.B. Marg, Dadar (W), Mumbai – 400 028. Tel. 4232 4232

CS : COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Databases

INDEX

Sr. No.	Contents	Topics	Pg. No.
1.	**Relational Model**		
	Notes	Introduction to DBMS	1
		Database System versus File System	3
		Database System Environment	6
		Database Languages	12
		Database System Structure	15
		Database Design	18
		Database Models	19
		Entity–Relationship Model	20
		Extended E–R Features	34
		The Relational Algebra	41
		The Tuple Relational Calculus	56
		LMR (Last Minute Revision)	62
	Assignment – 1	Questions	66
	Test Paper – 1	Questions	71
2.	**SQL**		
	Notes	Introduction to SQL	76
		Selections, Projections and Joins using SQL	77
		String Operations	79
		Set Operations	81
		Aggregate Functions	83
		Database Modifications	88
		Views	90
		Data Definition Language	92
		Constraints and Triggers	95
		LMR (Last Minute Revision)	100
	Assignment – 2	Questions	102
	Test Paper – 2	Questions	107

Sr. No.	Contents	Topics	Pg. No.
3.	**Normalization**		
	Notes	Functional Dependencies	111
		Keys of Relations	111
		Rules about Functional Dependencies	113
		Dependencies and Logical Implications	116
		Functional Dependencies and Keys	119
		Relational Database Design	121
		Normal Forms	122
		Multivalued Dependencies	130
		LMR (Last Minute Revision)	132
	Assignment – 3	Questions	134
	Test Paper – 3	Questions	138
4.	**Transaction Management**		
	Notes	Transaction	142
		Concurrent Executions	147
		Serial and Non–Serial Schedules	149
		Recoverability	158
		Concurrency Control	161
		Timestamps	166
		Validation–Based Protocols	167
		Deadlock Handling	169
		Recovery System	173
		Deferred Database Modification	176
		LMR (Last Minute Revision)	183
	Assignment – 4	Questions	185
	Test Paper – 4	Questions	189

Sr. No.	Contents	Topics	Pg. No.
5.	**File Structures**		
	Notes	Data Storage	196
		File Organization	201
		Indexing and Hashing	202
		Primary Index	203
		Multiple Levels of Index	204
		Secondary Index	207
		B^+ Trees	207
		Hashing	212
		Extendable Hashing	213
		Linear Hashing	215
		LMR (Last Minute Revision)	215
	Assignment–5	Questions	217
	Test Paper–5	Questions	220
Practice Problems		Questions	223
SOLUTIONS			
Assignments		Answer Key	233
		Model Solutions	234
Test Papers		Answer Key	247
		Model Solutions	248
Practice Problems		Answer Key	261
		Model Solutions	262

Topic 1 : Relational Model

INTRODUCTION TO DBMS

To better understand what drives the design of databases, you must understand the difference between data and information. Data are raw facts. The word "raw" is used to indicate that the facts have not yet been processed to reveal their meaning. Information is processed data on which decisions are based.

Ours is called the "information age." This term recognizes that the production of good, relevant, and timely information is the key to good decision making. In turn, good decision making is the key to business survival in a global market.

Key Points

- Data constitute the building blocks of information.
- Information is produced by processing data.
- Information is used to reveal the meaning of data.
- Good, relevant, and timely information is the key to good decision making.
- Good decision making is the key to organizational survival in a global environment.

Data Management

Clearly, timely and useful information requires good data. Such data must be generated properly, and it must be stored properly in a format that is easy to access and process. And, like any basic resource, the data environment must be managed carefully. Data management is a discipline that focuses on the proper generation, storage, and retrieval of data. Given the crucial role played by data, it should not surprise you that data management is a core activity for any business,government agency, service organization, or charity.

Typically, efficient data management requires the use of a computer database. A database is a shared, integrated computer structure that houses a collection of :

- End user data, that is, raw facts of interest to the end user.
- Metadata, or data about data, through which the data are integrated.

The metadata provide a description of the data characteristics and the set of relationships that link the data found within the database. In a sense, a database resembles a very well-organized electronic filing cabinet in which powerful software, known as a Database Management System (DBMS), helps manage the cabinet's contents.

Significance of Database Management System

The following points show why DBMSs are important :

1. Since data are so important, we must have a good way of managing such data. The DBMS helps make data management much more efficient and effective than it was before the DBMS became a reality.
2. The DBMS contains a query language that makes it possible to produce quick answers to ad hoc queries.
3. The DBMS helps create an environment in which end users have better access to more and better-managed data than they did before the DBMS became the data management standard.

Such access makes it possible for end users to respond quickly to changes in their environment. The availability of data, combined with the tools that transform data into usable information, empowers the end users to make quick and informed decisions that can make the difference between success and failure in the global economy.

4. Wider access to well-managed data promotes an integrated view of the organization's operations. Thus, it becomes much easier to see how actions in one segment of the company affect other segments. The DBMS environment promotes a clearer view of the big picture.
5. The probability of data inconsistency is greatly reduced in a properly designed database that is managed through a DBMS. Better data makes it possible to generate better information on which better decisions are based.



A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the database, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient.

Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information. In addition, the database system must ensure the safety of the information stored, despite system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.

The DBMS makes it possible to share the data in the database among multiple applications or users. Figure below illustrates that the DBMS stands between the database and the user(s). In effect, the DBMS serves as the intermediary between the user and the database by translating user requests into the complex code required to fulfill those requests. The DBMS hides much of the database's internal complexity from the application programs that use the database. The application program might be written by a programmer, using a programming language such as COBOL, or it might be created through a DBMS utility program.

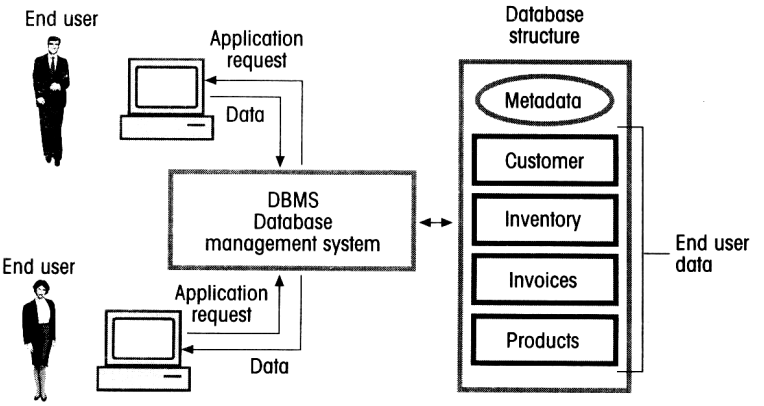


Fig. : The DBMS between end user and the Database

DATABASE SYSTEM VERSUS FILE SYSTEM

Although file systems are now largely obsolete, there are several good reasons for studying them in some detail :

1. File systems provide a useful historical perspective on the way we handle data.
2. A wise philosopher once pointed out that those who do not learn the lessons of history are doomed to repeat them. Some of the problems that plague file systems might be duplicated in database software if the users of the new software are unaware of the pitfalls of data management.
3. An understanding of the relatively simple characteristics of file systems makes the complexity of database design easier to understand.
4. If you intend to convert an obsolete file system to a database system, knowledge of the file system's basic limitations will be useful.

A typical file-processing system is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to the appropriate files. Before database management systems (DBMSs) came along, organizations usually stored information in such systems.

File System Data Management

Even the simplest data-retrieval task required extensive programming in a third-generation language (3GL). A 3GL requires the programmer to specify both what must be done and how it is to be done. Examples of 3GLs include Common Business-Oriented Language (COBOL), Beginner's All-purpose Symbolic Instruction Code (BASIC), and FORmula TRANslation (FORTRAN).

Programming in a 3GL can be a time-consuming, high-skill activity. Because even a simple file is quite different from the way the computer physically stores the data on disk, the programmer must be familiar with the physical file structure. Therefore, every file reference in a program requires the programmer to use complex coding to match the data characteristics and to define the precise access paths to the various file and system components. As file systems become more complex, the access paths become difficult to manage and tend to produce system malfunctions.

The need to write 3GL programs to produce even the simplest reports makes ad hoc queries impossible. Harried DP specialists or DP managers who work with mature file systems often receive numerous requests for new reports. They are often forced to say that the report will be ready "next week" or even "next month." If you need the information now, getting it next week or next month will not serve your information needs well.

As the number of files in the system expands, system administration becomes difficult, too. Each file must have its own file management system, composed of programs that allow the user to:

- Create the file structure.
- Add data to the file
- Delete data from the file.
- Modify the data contained in the file.
- List the file contents.

Even a simple file system containing 20 files requires $5 \times 20 = 100$ file management programs. If each of the files is accessed by 10 different reporting programs,an additional $20 \times 10 = 200$ programs must be written. Because ad hoc queries are not possible, the file reporting programs can multiply quickly. And because each department in the organization owns its data by creating its own files, the number of files can multiply rapidly.

Security features such as effective password protection, locking out parts of files or parts of the system itself, and other measures designed to safeguard data confidentiality are difficult to program and are, therefore usually omitted. Even when an attempt is made to improve system and data security, the security devices tend to be limited in scope and effectiveness.

The file system's structure and lack of security makes it difficult to pool data. The organizational structure promotes data ownership, thus promoting the storage of the same basic data in different locations. (Database professionals use the term **islands of information** to label such scattered data locations.) Because it is unlikely that data stored in different locations will always be updated consistently, the islands of information often contain different versions of the same data. The result is predictable although the file system is easily implemented in the beginning stages of computer data processing, a maturing file system is likely to spin out of control. The file system is simply not well suited to modern information requirements.

Structural and Data Dependence

A change in any file's structure, such as the addition or deletion of a field, requires the modification of all programs using that file. Such modifications are required because the file system exhibits **structural dependence,** that is, access to a file is dependent on its structure.

Even changes in file data characteristics, such as changing a field from integer to decimal, require changes in all programs that access the file. Because all data access programs are subject to change when any of the file's data characteristics change, the file system is said to exhibit **data dependence**.

The practical significance of data dependence is the difference between the data logical format (how the human being views the data) and the data physical format (how the computer "sees" the data). Therefore, any program that accesses a file system's file must not only tell the computer what to do, but also how to do it. Consequently, each program must contain lines that specify the opening of a specific file type, its record specification, and its field definitions. Data dependence thus helps to make the file system extremely cumbersome from a programming and data management point of view.

Disadvantages of File System

Keeping organizational information in a file-processing system has a number of major disadvantages:

1. **Data redundancy and inconsistency**

   Since different programmers create the files and application programs over a long period, the various files are likely to have different formats and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files).

For example, the address and telephone number of a particular customer may appear in a file that consists of savings-account records and in a file that consists of checking-account records. This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree. For example, a changed customer address may be reflected in savings-account records but not elsewhere in the system.

2. Difficulty in accessing data

Conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.

3. Data isolation

Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

4. Integrity problems

The data values stored in the database must satisfy certain types of consistency constraints. For example, the balance of a bank account may never fall below a prescribed amount (say, $25). Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files.

5. Atomicity problems

A computer system, like any other mechanical or electrical device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure. Consider a program to transfer $50 from account A to account B. If a system failure occurs during the execution of the program, it is possible that the $50 was removed from account A but was not credited to account B, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur. That is, the funds transfer must be atomic—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.

6. Concurrent-access anomalies

For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously. In such an environment, interaction of concurrent updates may result in inconsistent data. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been co-ordinated previously.

7. Security problems

Not every user of the database system should be able to access all the data. For example, in a banking system, payroll personnel need to see only that part of the database that has information about the various bank employees. They do not need access to information about customer accounts. But, since application programs are added to the system in an ad hoc manner, enforcing such security constraints is difficult.

The problems inherent in file systems make using the database system very desirable. Unlike the file system, with its many separate and unrelated files, the database consists of logically related data stored in a single data repository. Therefore, the database represents a change in the way end user data are stored, accessed, and managed. The database's DBMS, provides numerous advantages over file system management by making it possible to eliminate most of the file system's data inconsistency, data anomalies, data dependency, and structural dependency problems. Better yet, the current generation of DBMS software stores not only the data structures, but also the relationships between the database components, all in a central location. The current generation of DBMS software also takes care of defining, storing, and managing all the required access paths to those components.

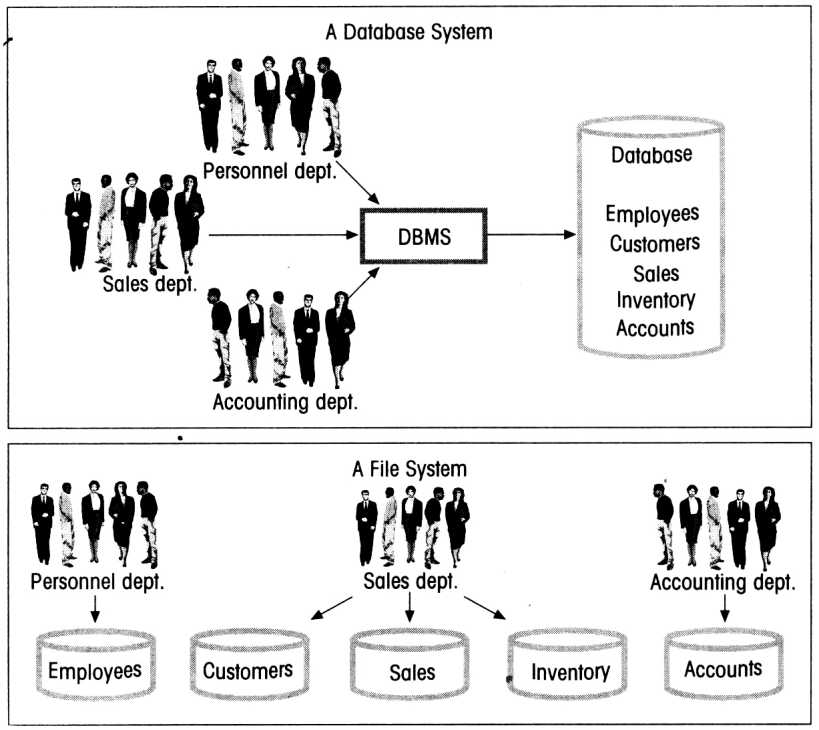


Fig. : Contrasting Database and File systems

DATABASE SYSTEM ENVIRONMENT

The term database system refers to an organization of components that define and regulate the collection, storage, management, and use of data within a database environment. From a general management point of view, the database system is composed of the five major parts as shown in figure below : Hardware, Software, People, Procedures, and Data.

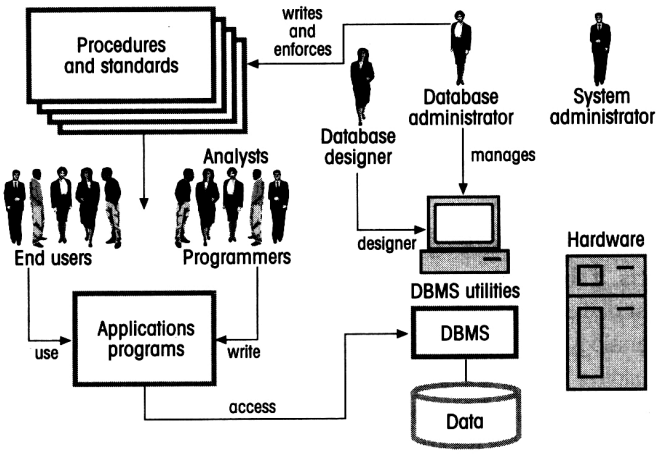


Fig. : Database System Environment

1. Hardware

Hardware identifies all the system's physical devices. The database system's main and most easily identified hardware component is the computer, which might be a microcomputer, a minicomputer, or a mainframe computer. The hardware also includes all of the computer peripherals, which are the physical devices that control computer input and output. In other words, peripherals include keyboards, mice, modems, printers, etc. Peripherals also include any electronic devices that are used to connect two or more computers, thereby producing a computer network. Such networks are especially important for databases that store data that are likely to be accessed from remote locations, such as airplane reservation systems, automatic teller machines, and so on.

2. Software

Software refers to the collection of programs used by computers within the database system. Although the most readily identified software is the DBMS itself, it takes three types of software to make the database system function fully: operating systems software, DBMS software, and application programs and utilities.

- **Operating systems software** manages all hardware components and makes it possible for all other software to run on the computers. Examples of operating systems software include the disk operating system (DOS), OS/2, and Windows used by microcomputers, UNIX and VMS used by minicomputers, and MVS used by IBM mainframe computers.
- **DBMS software** manages the database within the database system. Some examples of DBMS software include Microsoft's Access and SQL Server, Oracle Corporation's Oracle, and IBM's DB2.
- **Application programs and utilities software** are used to access and manipulate the data in the DBMS and to manage the computer environment in which data access and manipulation take place. Application programs are most commonly used to access the data found within the database to generate reports, tabulations, and other information to facilitate decision making. Utilities are the software tools used to help manage the database system's computer components.

3. People

This component includes all users of the database system. On the basis of primary job functions, we can identify five types of users in a database system: systems administrators, database administrators, database designers, systems analysts / programmers, and end users. The members of each user type perform both unique and complementary functions.

- **Systems administrators** oversee the database system's general operations.
- **Database administrators**, also known as DBAs, manage the use of DBMS's and ensure that the database is functioning properly. The DBA's role is sufficiently important to warrant a detailed exploration in the Database Administration.
- **Database designers** design the database structure. They are, in effect, the database architects. If the database design is poor, even the best application programmers and the most dedicated DBAs will fail to produce a useful database environment. To use an old analogy, the finest bricklayers and carpenters will not be able to produce a good building from a bad blueprint. Because organizations strive to optimize their data resources, the (relatively new) job description of database designer has expanded to cover new dimensions and growing responsibilities.
- **Systems analysts and programmers** design and implement the application programs. They design and create the data entry screens, reports, and procedures through which end–users access and manipulate the database's data.
- **End users** are the people who use the application programs to run the organization's daily operations. For example, sales clerks, supervisors, managers, and directors are all classified as end–users. High-level end users employ the information obtained from the database to make tactical and strategic business decisions.

4. Procedures

Procedures are the instructions and rules that govern the design and use of the database system. Procedures are a critical, although occasionally forgotten, component of the system. Procedures play an important role in a company, because they enforce the standards by which business is conducted within the organization and with customers. Procedures also are used to ensure that there is an organized way to monitor and audit both the data that enter the database and the information that is generated through the use of such data.

5. Data

The word "data" covers the collection of facts stored in the database. Because data are the raw material from which information is generated, the determination of which data are to be entered into the database and how such data are to be organized is a vital part of the database designer's job.

DBMS FUNCTIONS

A DBMS performs several important functions that guarantee the integrity and consistency of the data in the database. Most of these functions are transparent to end users, and most can be achieved only through the use of a DBMS.

1. Data dictionary management

The DBMS requires that definitions of the data elements and their relationships (metadata) be stored in a **data dictionary**. In turn, all programs that access the data in the database work through the DBMS. The DBMS uses the data dictionary to look up the required data component structures and relationships, thus relieving us from having to code such complex relationships in each program. Additionally, any changes made in a database structure are automatically recorded in the data dictionary, thereby freeing us from having to modify all the programs that access the changed structure. In other words, the DBMS provides data abstraction, and it removes structural and data dependency from the system.

2. Data storage management

The DBMS creates the complex structures required for data storage, thus relieving us from the difficult task of defining and programming the physical data characteristics. A modern DBMS system provides storage not only for the data, but also for related data entry forms or screen definitions, report definitions, data validation rules, procedural code, structures to handle video and picture formats, and so on.

3. Data transformation and presentation

The DBMS transforms entered data to conform to the data structures that are required to store the data. Therefore, the DBMS relieves us of the chore of making a distinction between the data logical format and the data physical format. By maintaining data independence, the DBMS translates logical requests into commands that physically locate and retrieve the requested data. That is, the DBMS formats the physically retrieved data to conform to the user's logical expectations. In other words, a DBMS provides application programs with software independence and data abstraction.

4. Security management

The DBMS creates a security system that enforces user security and data privacy within the database. Security rules determine which users can access the database, which data items each user may access, and which data operations (read, add, delete, or modify) the user may perform. This is especially important in multiuser database systems where many users can access the database simultaneously.

5. Multiuser access control

The DBMS creates the complex structures that allow multiuser access to the data. In order to provide data integrity and data consistency, the DBMS uses sophisticated algorithms to ensure that multiple users can access the database concurrently without compromising the integrity of the database.

6. Backup and recovery management

The DBMS provides backup and data recovery procedures to ensure data safety and integrity. Current DBMS systems provide special utilities that allow the DBA to perform routine and special backup and restore procedures. Recovery management deals with the recovery of the database after a failure, such as a bad sector in the disk or a power failure. Such capability is critical to the preservation of the database's integrity.

7. Data integrity management

The DBMS promotes and enforces integrity rules to eliminate data integrity problems, thus minimizing data redundancy and maximizing data consistency. The data relationships stored in the data dictionary are used to enforce data integrity. Ensuring data integrity is especially important in transaction-oriented database systems.

8. Database access languages and application programming interfaces

The DBMS provides data access via a query language. A query language is a nonprocedural language– that is, one that lets the user specify what must be done without having to specify how it is to be done. The DBMS's query language contains two components: a data definition language (DDL) and a data manipulation language (DML.) The DDL defines the structures in which the data are housed, and the DML allows end users to extract the data from the database. The DBMS also provides data access to programmers via procedural (3GL) languages such as COBOL, C, PASCAL, BASIC, and others. The DBMS also provides administrative utilities used by the DBA and the database designer to create, implement, monitor, and maintain the database.

9. Database communication interfaces

Current-generation DBMS provide special communications routines designed to allow the database to accept end users request within a computer network environment. In fact, database communications capabilities are an essential feature of the modern DBMS. For example, the DBMS might provide communication functions to access the database through the Internet, using Internet browsers such as Netscape or Explorer as the front end.

Data Abstraction

A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides the details of how the data are stored and maintained.

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-systems users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

1. Physical level

The physical level is the lowest level of abstraction that describes how the data are actually stored. The physical level describes complex low-level data structures in detail. At the physical level a record can be described as a block of consecutive storage locations (for example, words or bytes). The language compiler hides this level of detail from programmers. Similarly, the database system hides many of the lowest-level storage details from database programmers. Database administrators, on the other hand, may be aware of certain details of the physical organization of the data.

2. Logical level

The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

3. View level

The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. At the view level, computer users see a set of application programs that hide details of the data types. Many users of the database system do not need all this information; instead they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

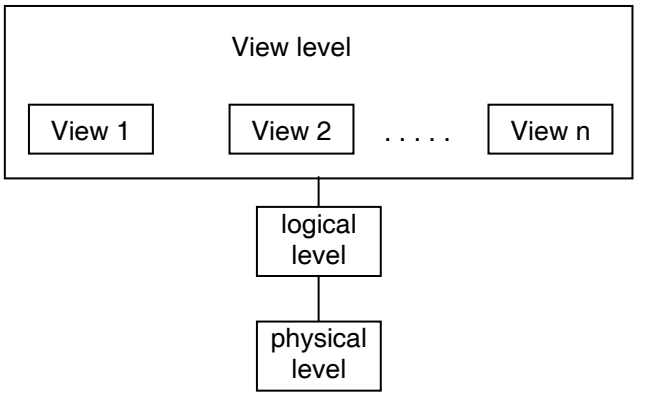


Fig. : The three level of data abstraction

DATABASE LANGUAGES



A database system provides a data definition language to specify the database schema and a data manipulation language to express database queries and updates. In practice, the data definition and data manipulation languages are not two separate languages, instead they simply form parts of a single database language, such as the widely used SQL language.

Data - Definition Language

We specify a database schema by a set of definitions expressed by a special language called a **data-definition language (DDL).**

For instance, the following statement in the SQL language defines the account table:

create table *account*
(account-number ***char****(10),*
balance ***integer****)*

Execution of the above DDL statement creates the account table. In addition, it updates a special set of tables called the data dictionary or data directory.

A data dictionary contains metadata – that is, data about data. The schema of a table is an example of **metadata**. A database system consults the data dictionary before reading or modifying actual data.

We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a data storage and definition language. These statements define the implementation details of the database schemas, which are usually hidden from the users.

The data values stored in the database must satisfy certain consistency constraints. For example, suppose the balance on an account should not fall below $100. The DDL provides facilities to specify such constraints. The database systems check these constraints every time the database is updated.

Data - Manipulation Languages

Data manipulation is

- The retrieval of information stored in the database
- The insertion of new information into the database
- The deletion of information from the database
- The modification of information stored in the database

A data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model. There are basically two types :

- Procedural DMLs require a user to specify what data are needed and how to get those data.
- Declarative DMLs (also referred to as nonprocedural DMLs) require a user to specify what data are needed without specifying how to get those data.

Declarative DMLs are usually easier to learn and use than the procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data. The DML component of the SQL language is nonprocedural.

A query is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a query language. Although technically incorrect, it is common practice to use the terms query language and data manipulation language synonymously.

This query in the SQL language finds the name of the customer whose customer-id is 192:

select *customer.customer-name*
from *customer*
where *customer.customer-id = 192*

The query specifies that those rows from the table customer where the customer-id is 192 must be retrieved, and the customer-name attribute of these rows must be displayed. Queries may involve information from more than one table.

There are a number of database query languages in use, either commercially or experimentally. One of the most widely used query language is SQL.

Database Access from Application Programs

Application programs are programs that are used to interact with the database. Application programs are usually written in a host language such as Cobol, C, C++ or Java. Examples in a banking system are programs that generate payroll checks, debit accounts, credit accounts, or transfer funds between accounts.

To access the database, DML statements need to be executed from the host language. There are two ways to do this :

- By providing an application program interface (set of procedures) that can be used to send DML and DDL statements to the database, and retrieve the results.

  The Open Database Connectivity (ODBC) standard defined by Microsoft for use with the C language is a commonly used application program interface standard. The Java Database Connectivity (JDBC) standard provides corresponding features to the Java language.

- By extending the host language syntax to embed DML calls within the host language program. Usually, a special character prefaces DML calls, and a preprocessor, called the DML precompiler, converts the DML statements to normal procedure calls in the host language.

Database Users and Administrators

A primary goal of a database system is to retrieve information from and store new information in the database. People who work with a database can be categorized as database users or database administrators.

Database Users and User Interfaces :

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for the different types of users.

1. **Naive users**

   Naive users are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously. For example, a bank teller who needs to transfer $50 from account A to account B invokes a program called transfer. This program asks the teller for the amount of money to be transferred, the account from which the money is to be transferred, and the account to which the money is to be transferred.

   The typical user interface for naive users is a forms interface, where the user can fill in appropriate fields of the form. Naive users may also simply read reports generated from the database.

2. **Application programmers**

   Application programmers are computer professionals who write application programs. Application programmers can choose from many tools to develop user interfaces. Rapid application development (RAD) tools are tools that enable an application programmer to construct forms and reports without writing a program. There are also special types of programming languages that combine imperative control structures (for example, for loops, while loops and if-then-else statements) with statements of the data manipulation language. These languages, sometimes called fourth-generation languages, often include special features to facilitate the generation of forms and the display of data on the screen. Most major commercial database systems include a fourth-generation language.

3. **Sophisticated users**

   Sophisticated users interact with the system without writing programs. Instead, they form their requests in a database query language. They submit each such query to a query processor, whose function is to break down DML statements into instructions that the storage manager understands. Analyst who submits queries to explore data in the database fall in this category.

4. **Specialized users**

   Specialized users are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework. Among these applications are computer-aided design systems, knowledge-base and expert systems, systems that store data with complex data types (for example, graphics data and audio data), and environment-modeling systems.

Database Administrator

One of the main reasons for using DBMS is to have central control of both the data and the programs that access those data. A person who has such central control over the system is called a **database administrator (DBA)**. The functions of a DBA include:

1. **Schema definition.** The DBA creates the original database schema by executing a set of data definition statements in the DDL.
2. **Storage structure and access-method definition.**
3. **Schema and physical-organization modification.** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.

4. **Granting of authorization for data access.** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.

5. **Routine maintenance.** Examples of the database administrator's routine maintenance activities are:
   - Periodically backing up the database, either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
   - Ensuring that enough free disk space is available for normal operations and upgrading disk space as required.
   - Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

DATABASE SYSTEM STRUCTURE

A database system is partitioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be broadly divided into the storage manager and the query processor components.

The storage manager is important because databases typically require a large amount of storage space. Corporate databases range in size from hundreds of gigabytes to, for the largest databases, terabytes of data. A gigabyte is 1000 megabytes (1 billion bytes), and a terabyte is 1 million megabytes (1 trillion bytes). Since the main memory of computers cannot store this much information, the information is stored on disks. Data are moved between disk storage and main memory as needed. Since the movement of data to and from disk is slow relative to the speed of the central processing unit, it is imperative that the database system structure the data so as to minimize the need to move data between disk and main memory.

The query processor is important because it helps the database system simplify and facilitate access to data. High-level views help to achieve this goal; with them, users of the system are not to be burdened unnecessarily with the physical details of the implementation of the system. However, quick processing of updates and queries is important. It is the job of the database system to translate updates and queries written in a nonprocedural language, at the logical level, into an efficient sequence of operations at the physical level.

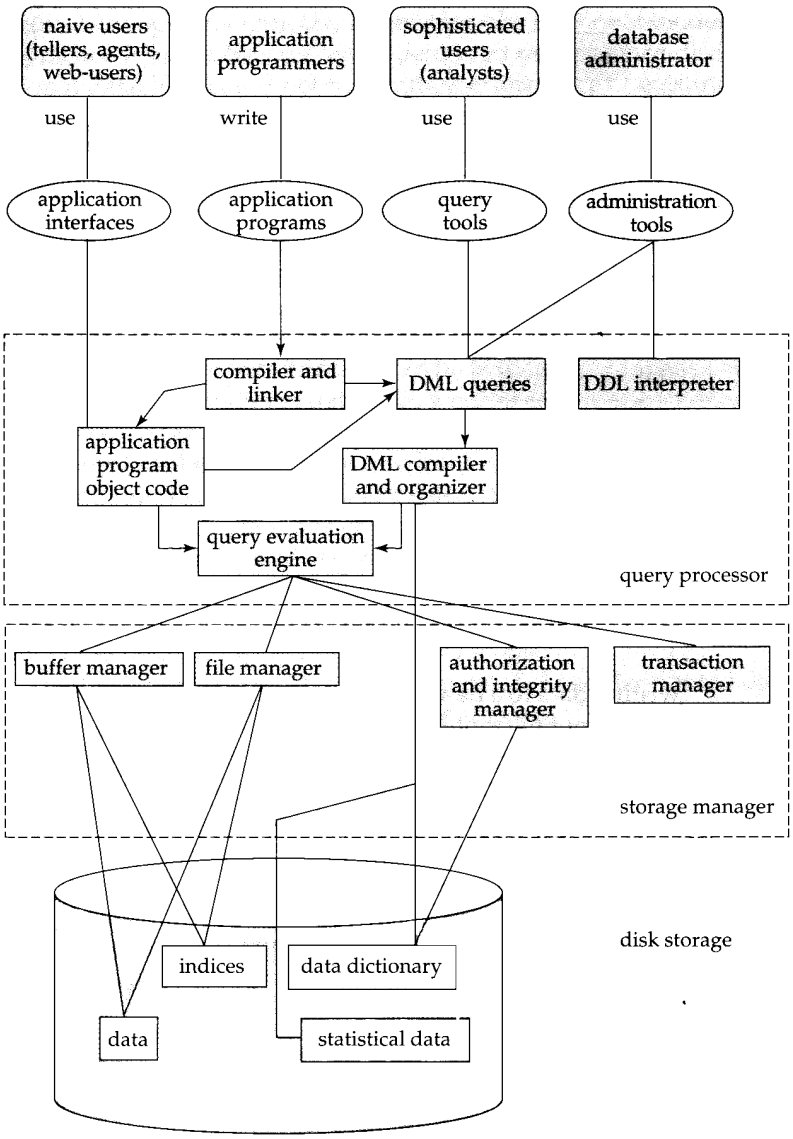


Fig. : Database System Structure

Storage Manager

A storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system. The storage manager is responsible for the interaction with the file manager. The raw data are stored on the disk using the file system, which is usually provided by a conventional operating system. The storage manager translates the various DML statements into low-level file-system commands. Thus, the storage manager is responsible for storing, retrieving, and updating data in the database. The storage manager components include:

1. **Authorization and integrity manager**, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.

2. **Transaction manager**, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.

3. **File manager**, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.

4. **Buffer manager**, which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory. The buffer manager is a critical part of the database system, since it enables the database to handle data sizes that are much larger than the size of main memory.

The storage manager implements several data structures as part of the physical system implementation:

1. **Data files**, which store the database itself.
2. **Data dictionary**, which stores metadata about the structure of the database, in particular the schema of the database.
3. **Indices**, which provide fast access to data items that hold particular values.

The Query Processor

The query processor components include

1. **DDL interpreter,** which interprets DDL statements and records the definitions in the data dictionary.

2. **DML compiler,** which translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.

   A query can usually be translated into any of a number of alternative evaluation plans that all give the same result. The DML compiler also performs query optimization, that is, it picks the lowest cost evaluation plan from among the alternatives.

3. **Query evaluation engine**, which executes low-level instructions generated by the DML compiler.

APPLICATION ARCHITECTURES

Most users of a database system today are not present at the site of the database system, but connect to it through a network. We can therefore differentiate between client machines, on which remote database users work and server machines, on which the database system runs.

Database applications are usually partitioned into two or three parts, as in figure below. In a two-tier architecture, the application is partitioned into a component that resides at the client machine, which invokes database system functionality at the server machine through query language statements. Application program interface standards like ODBC and JDBC are used for interaction between the client and the server.

In contrast, in a three-tier architecture, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an application server, usually through a forms interface. The application server in turn communicates with a database system to access data. The business logic of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients. Three-tier applications are more appropriate for large applications and for applications that run on the World Wide Web.

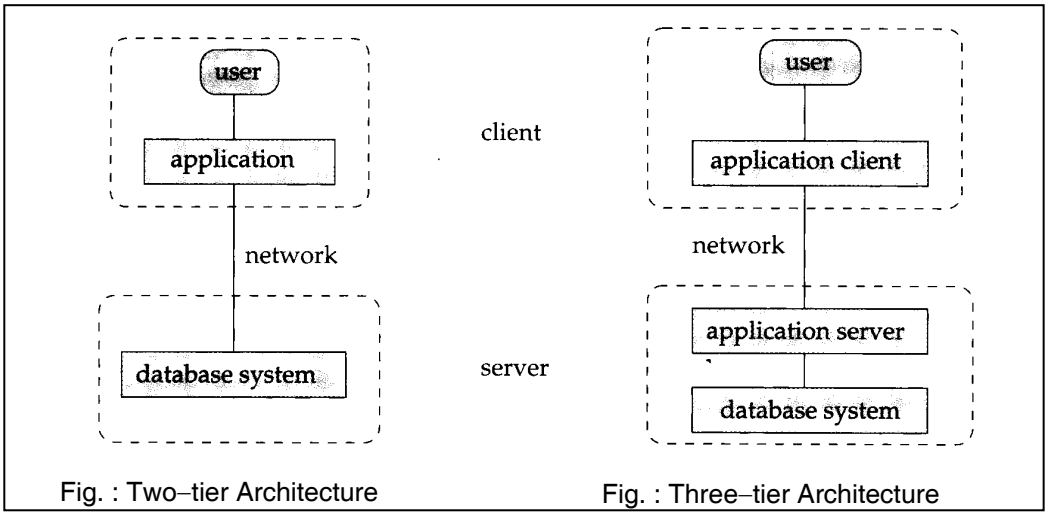


Fig. : Two–tier Architecture

Fig. : Three–tier Architecture

DATABASE DESIGN

Keep in mind that the DBMS is the database software you buy commercially; you do not have the option of making design changes to it. Therefore, when we speak of database design, we mean the design of the database structure that will be used to store and manage data, rather than the design of the DBMS software. Once the database design is completed, the DBMS handles all the complicated activities required to translate the designer's view of the structures into structures that are usable to the computer.

The availability of a DBMS makes it possible to tackle far more sophisticated uses of the data resources, if the database is designed to make use of that available power. The kinds of data structures created within the database and the extent of the relationships among them play a powerful role in determining how effective the DBMS is. Therefore, database design becomes a crucial activity in the database environment.

Database design is made much simpler when you use models. Models are simplified abstractions of real-world events or conditions. For example, such abstractions will enable us to explore the characteristics of entities and the relationships that can be created among such entities. If the models are not logically sound, the database designs derived from them will not deliver the database system's promise of effective information drawn from an efficient database. Good models yield good database designs that are the basis for good applications. Conversely, you cannot expect to build good applications on bad database design based on poor models.

Instances and Schemas

Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. The overall design of the database is called the database schema. Schemas are changed infrequently, if at all.

The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an instance of a database schema.

Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas that describe different views of the database.

Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit physical data independence if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

DATABASE MODELS



A database model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

Database models can be grouped into two categories: conceptual models and implementation models.

1. The **conceptual model** focuses on the logical nature of the data representation. Therefore, the conceptual model is concerned with what is represented in the database, rather than with how it is represented. Conceptual models include the entity relationship (E-R) model.
2. In contrast to the conceptual model, an **implementation model** places the emphasis on how the data are represented in the database or on how the data structures are implemented to represent what is modeled. Implementation models include the hierarchical database model, the network database model, the relational database model and the object-oriented database model.

In this part, we will study two data models – the **entity-relationship model** and the **relational model**.

The entity-relationship (E-R) model is a high-level data model. It is based on a perception of a real world that consists of a collection of basic objects, called entities, and of relationships among these objects.

The relational model is a lower-level model. It uses a collection of tables to represent both data and the relationships among those data. Its conceptual simplicity has led to its widespread adoption; today a vast majority of database products are based on the relational model. Designers often formulate database schema design by first modeling data at a high level, using the E-R model, and then translating it into the relational model.

ENTITY–RELATIONSHIP MODEL

The entity-relationship (E-R) data model perceives the real world as consisting of basic objects, called entities, and relationships among these objects. It was developed to facilitate database design by allowing specification of an enterprise schema, which represents the overall logical structure of a database. The E-R data model is one of several semantic data models; the semantic aspect of the model lies in its representation of the meaning of the data. The E-R model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the E-R model.

E–R models are normally represented in an entity relationship diagram (ERD). The ERD uses graphic representation to model the database components.

E–R MODEL COMPONENTS

1. Entity

An entity is a "thing" or "object" in the real world that is distinguishable from all other objects. For example, each person in an enterprise is an entity. An entity has a set of properties, and the values for some set of properties may uniquely identify an entity. For instance, a person may have a person-id property whose value uniquely identifies that person. An entity may be concrete, such as a person or a book, or it may be abstract, such as a loan, or a holiday, or a concept.



An entity set is a set of entities of the same type that share the same properties, or attributes.

The set of all persons who are customers at a given bank, for example, can be defined as the entity set *customer*.

An entity at the E- R modeling level actually refers to the entity set and not to a single entity occurrence. In other words, the word "entity" in the E-R model corresponds to a table and not to a row in the relational environment. The E-R model refers to a specific table row as an **entity instance** or **entity occurrence**. An entity is represented by a **rectangle** containing the entity's name. The entity name, a **noun**, is usually written in capital letters.

2. Attributes

An entity is represented by a set of attributes. Attributes are descriptive properties possessed by each member of an entity set. The designation of an attribute for an entity set expresses that the database stores similar information concerning each entity in the entity set; however, each entity may have its own value for each attribute. For instance, a particular customer entity may have the value 321-12 for customer-id, the value Sandeep for customer-name, the value M.G. road for customer-street and the value Harrison for customer-city.

For each attribute, there is a set of permitted values, called the **domain**, or **value set**, of that attribute. The domain of attribute customer-name might be the set of all text strings of a certain length.

Formally, an attribute of an entity set is a function that maps from the entity set into a domain. Since an entity set may have several attributes, each entity can be described by a set of (attribute, data value) pairs, one pair for each attribute of the entity set.

Attributes are represented by ovals and are connected to the entity with a line. Each oval contains the name of the attribute it represents. For example, the STUDENT entity includes the attributes STU_LNAME, STU_FNAME and STU_INITIAL as shown in Figure.

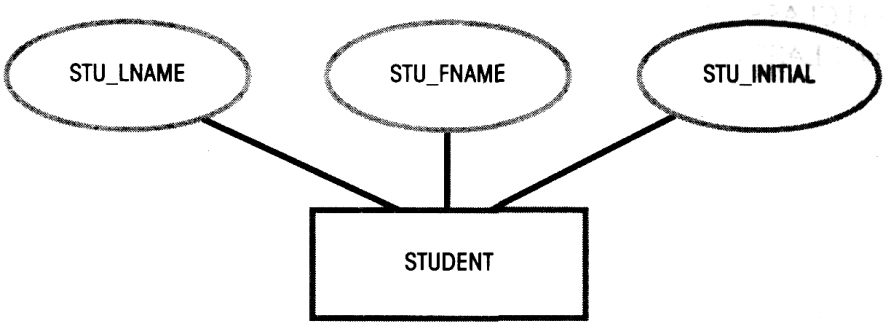


Fig. : Attributes of Student entity

Attributes may share a domain. For instance, a student address and a professor address share the same domain of all possible addresses.

Attributes types

An attribute, as used in the E-R model, can be characterized by the following attribute types.

Simple or composite attributes

A composite attribute is an attribute that can be further subdivided to yield additional attributes. For example, the attribute ADDRESS can be subdivided into street, city, state and zip code. Similarly, the attribute PHONE_NUMBER can be subdivided into area code and exchange number. A simple attribute cannot be subdivided. For example, age, sex and marital status would be classified as simple attributes. To facilitate detailed queries, it is usually appropriate to change composite attributes into a series of simple attributes.

Single-valued and multivalued attribute

A single-valued attribute can have only a single value. For example, a person can have only one Social Security number, and a manufactured part can have only one serial number. Keep in mind that a single-valued attribute is not necessarily a simple attribute. For instance, the part's serial number SE-08-02-189935 is single-valued, but it is a composite attribute because it can be subdivided into the region in which the part was produced, SE, the plant within that region, 08, the shift within the plant, 02 and the part number 189935.

Multivalued attributes can have many values. For instance, a person may have several college degrees, or a household may have several different phones, each with its own number. Similarly, a car's color may be subdivided into colors of roof, body, and trim. In an E-R model the multivalued attributes as shown by double line connecting the attributes to the entity.

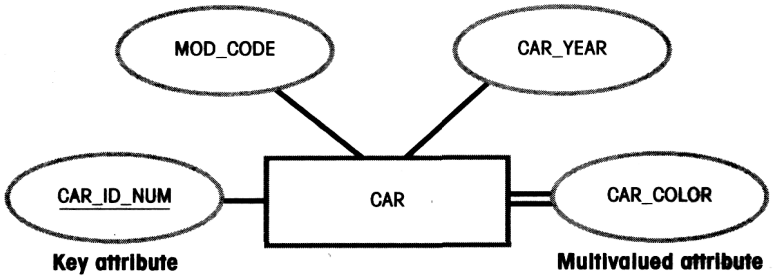


Fig. : Multivalued attribute

Although the conceptual model can handle multivalued attributes, you should not implement them in the relational DBMS. So, if multivalued attributes exist, the designer must decide on one of two possible courses of action:

1. Within the original entity, create several new attributes, one for each of the original multivalued attribute's components. For example, we can split the CAR entity's attribute CAR_COLOR to create the new attributes CAR_TOPCOLOR, CAR_BODYCOLOR, and CAR_TRIMCOLOR, shown in figure below, and assign them to the CAR entity.

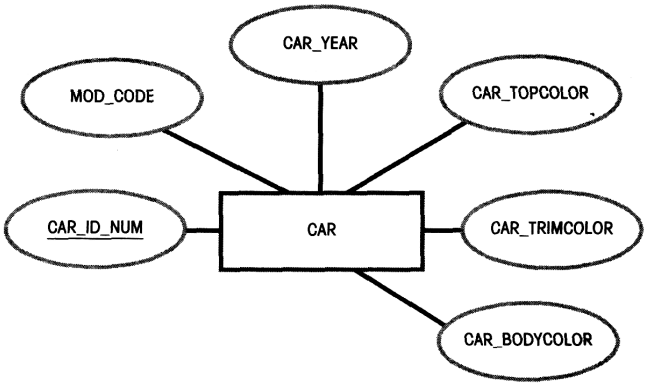


Fig. : Splitting of multiple attributes into new attributes

2. Create a new entity composed of the original multivalued attribute's components (see figure below). The new (independent) COLOR entity is then related to the original CAR entity in a 1: M relationship.

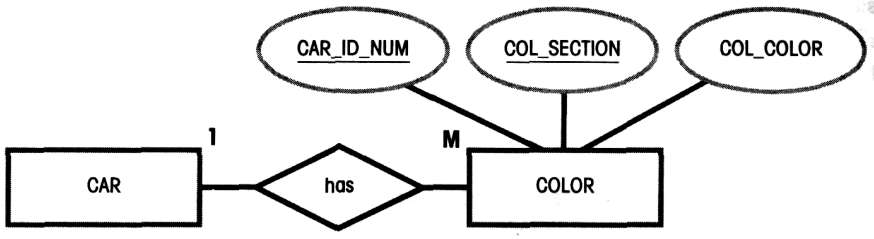


Fig. : A new entity set composed of a multivalued attribute's components

Derived attributes

The value for this type of attribute can be derived from the values of other relative attributes entities.

A derived attribute need not be physically stored within the database; instead, it is derived by using an algorithm. For example, a person's AGE attribute can be derived by subtracting the date of birth (DOB) from the current date. Similarly, the total cost of an order can be derived by multiplying the quantity ordered by the unit price. Or the estimated average speed can be derived by dividing trip distance by the time spent en route. A derived attribute is indicated by a **dashed line** connecting the attribute and the entity (see figure below). The decision to use derived attributes depends on processing requirements and the constraints placed on a particular application. The designer should be able to balance the design in accordance with such constraints.

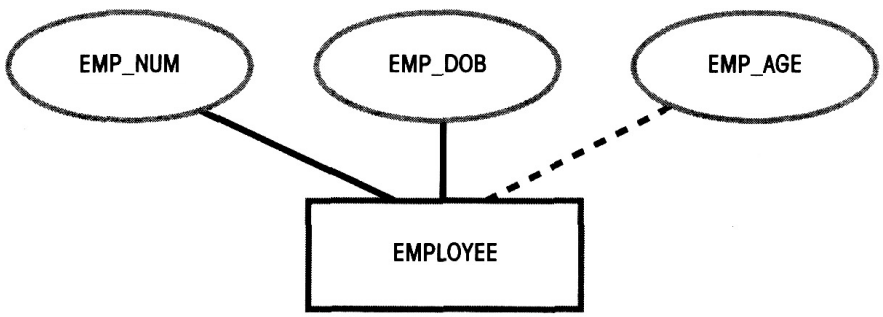


Fig. : A Derived Attribute

3. Relationships

A relationship describes an association among data. Most relationships describe associations between two entities.

Conceptual models use three types of relationships to describe associations among data: one-to-many, many-to-many, and one-to-one.

Relationships are represented by a **diamond** connected to the related entities. The name of the relationship, an active or passive verb, is written inside the diamond.

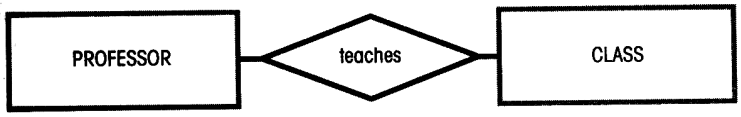


Fig. : An Entity Relationship

Connectivity

ERD models use the term connectivity to label the relationship classifications.

The connectivities are written next to each entity box.

Database designers usually use the shorthand notations 1: M, M : N, and 1:1 for them, respectively.

The following examples illustrate the distinctions among the three.

1. **One-to-many relationship.**

   A painter paints many different paintings, but each one of them is painted by only that painter. Thus the painter (the "one") is related to the paintings (the "many"). Therefore, database designers label the relationship "PAINTER paints PAINTING" as 1:M.

2. **Many-to-many relationship.**

   An employee might learn many job skills, and each job skill might be learned by many employees. Database designers label the relationship "EMPLOYEE learns SKILL" as M:N.

3. **One-to-one relationship.**

   A retail company's management structure may require that each one of its stores be managed by a single employee. In turn, each store manager – who is an employee – only manages a single store. Therefore, the relationship "EMPLOYEE manages STORE" is labeled 1:1.

We have illustrated the use of ERDs in figure below using the relationships discussed above.

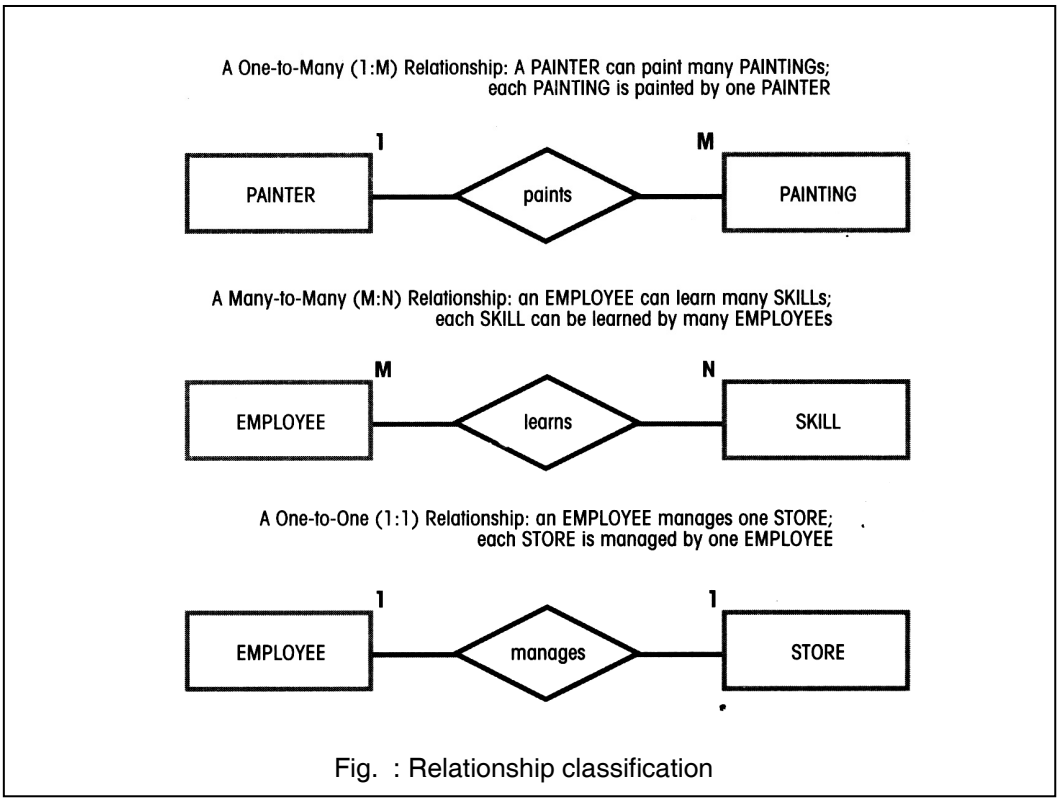


Fig. : Relationship classification

Types of Relationships

A relationship's degree indicates the number of associated entities or participants. A unary relationship exists when an association is maintained within a single entity. A binary relationship exists when two entities are associated. A ternary relationship exists when three entities are associated. Although higher degrees exist, they are rare and are not specifically named. (For example, an association of four entities is simply described as a four-entity relationship.) We have shown a few types of relationships in figure.

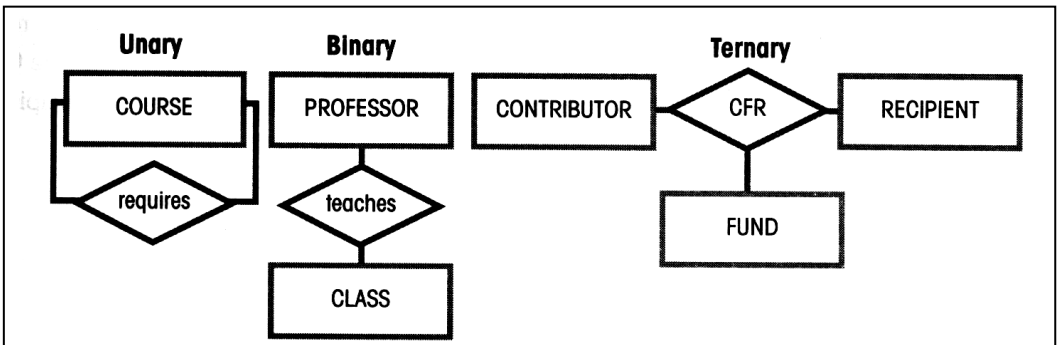


Fig. : Three types of Relationships

In the case of the unary relationship shown in Figure above, a course within the COURSE entity is a prerequisite for another course within that entity. In this case, the existence of a course prerequisite means that a COURSE has a relationship with itself. Such a relationship is known as a **recursive relationship.**

Binary relationships are the most common. In fact, to simplify the conceptual design, most higher-order (ternary and higher) relationships are decomposed into appropriate equivalent binary relationships whenever possible.

Relationship Sets

A relationship set is a set of relationships of the same type. Formally, it is a mathematical relation on $n \geq 2$ (possibly nondistinct) entity sets. If $E_1, E_2, \ldots E_n$ are entity sets, then a relationship set R is a subset of $\{(e_1, e_2, ..., e_n) \mid e_1 \in E_1, e_2 \in E_2, ..., e_n \in E_n\}$

where $(e_1, e_2, ..., e_n)$ is a relationship.

Consider the two entity sets *customer* and *loan*. We define the relationship set borrower to denote the association between customers and the bank loans that the customers have. Figure below depicts this association.

Customer id	*customer name*	*Customer city*
055–28	Deepak	Sion
244–80	Amit	Malad
134–35	Jumana	Kurla
936–11	Shalu	Dadar
335–57	Manish	Mulund

loan–number	*loan–amount*
L–128	3000
L–129	4000
L–133	7500
L–166	9600
L–178	10000

customer *loan*

Fig.: Relationship set *borrower*

The association between entity sets is referred to as participation; that is, the entity sets $E_1, E_2, \ldots E_n$ participate in relationship set R. A **relationship instance** in an E-R schema represents an association between the named entities in the real-world enterprise that is being modeled.

The function that an entity plays in a relationship is called that entity's role.

CONSTRAINTS

An E-R enterprise schema may define certain constraints to which the contents of a database must conform. In this section, we examine mapping cardinalities and participation constraints, which are two of the most important types of constraints.

1. Mapping Cardinalities

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.

Mapping cardinalities are most useful in describing binary relationship sets, although they can contribute to the description of relationship sets that involve more than two entity sets. In this section, we shall concentrate on only binary relationship sets.

For a binary relationship set R between entity sets A and B, the mapping cardinality must be one of the following:

- One to one: An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.
- One to many: An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.
- Many to one: An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.
- Many to many: An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A.

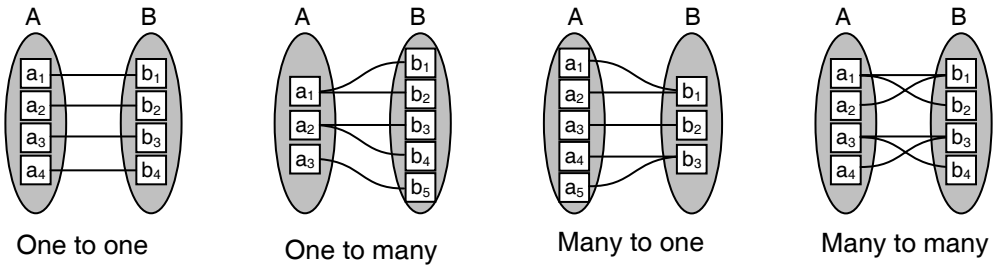


Fig. : Mapping Cardinalities

Cardinality

Cardinality expresses the specific number of entity occurrences associated with one occurrence of the related entity. The actual number of associated entities usually is a function of an organization's policy. For example, the Oxford College administration limits the professor to teaching a maximum of three classes per semester. Therefore, the cardinality rule governing the PROFESSOR-CLASS association is expressed as, "One professor teaches up to three classes per semester." Similarly, the Oxford College administration may decide to limit a student to a maximum of six classes per semester, and class size may be limited to 35 students. The cardinality is indicated by placing the appropriate numbers beside the entities, as shown in figure below.

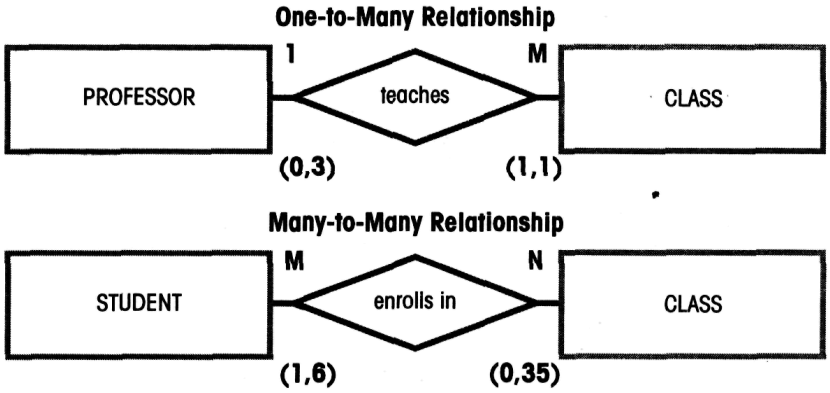


Fig. : Cardinality in an ERD

2. Participation Constraints

The participation of an entity set E in a relationship set R is said to be **total** if every entity in E participates in at least one relationship in R. If only some entities in E participate in relationship in R, the participation of entity set E in relationship R is said to be **partial**.

A participating entity in a relationship is either optional or mandatory. The participation is optional if one entity occurrence does not require a corresponding entity occurrence in a particular relationship. For example, suppose that Tiny College employs some professors who conduct research without teaching classes. If we examine the relationship "PROFESSOR teaches CLASS," it is quite possible for a PROFESSOR not to teach a CLASS. Therefore, CLASS is optional to PROFESSOR. On the other hand, a CLASS must be taught by a PROFESSOR. Therefore, PROFESSOR is mandatory to CLASS.

An optional entity is shown by drawing a small circle (o) on the side of the optional entity, as shown in Figure.

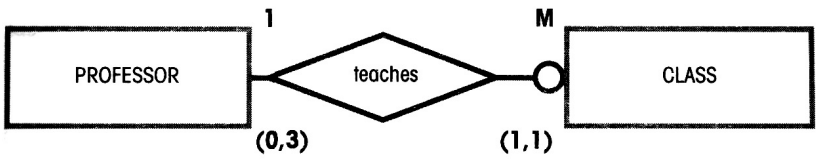


Fig. : An ERD with an optional entity

KEYS

We must have a way to specify how entities within a given entity set are distinguished. Conceptually, individual entities are distinct; from a database perspective, however, the difference among them must be expressed in terms of their attributes.

Therefore, the values of the attribute of an entity must be such that they can uniquely identify the entity. In other words, no two entities in an entity set are allowed to have exactly the same value for all attributes.



A key allows us to identify a set of attributes that suffice to distinguish entities from each other. Keys also help to uniquely identify relationships and thus distinguish relationships from each other.

1. Entity Sets

(i) *Superkey* :

A superkey is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set. For example, the customer-id attribute of the entity set customer is sufficient to distinguish one customer entity from another. Thus, customer-id is a superkey. Similarly, the combination of customer-name and customer-id is a superkey for the entity set customer. The customer-name attribute of customer is not a superkey, because several people might have the same name.

(ii) *Candidate key* :

A Candidate key is a superkey for which no proper subset is a superkey. In order words, candidate keys are minimal superkeys.

It is possible that several distinct sets of attributes could serve as a candidate key. Suppose that a combination of customer-name and customer-street is sufficient to distinguish among members of the customer entity set. Then, both {customer-id} and {customer-name, customer-street} are candidate keys. Although the attributes customer-id and customer-name together can distinguish customer entities, their combination does not form a candidate key, since the attribute customer-id alone is a candidate.

(iii) *Primary key* :

A primary key is a candidate key that is chosen by the database designer as the principle means of identifying entities within an entity set. In E–R diagram, a primary key is underlined by a solid line.

Note, incidentally, that a primary key is a super key as well as candidate key.

2. Relationship Sets :

The primary key of an entity set allows us to distinguish among the various entities of the set. We need a similar mechanism to distinguish among the various relationships of a relationship set. Let R be a relationship set involving entity sets E_1, E_2, ... E_n.

Let primary-key (E_i) denote the set of attributes that forms the primary key for entity set E_i. Assume for now that the attribute names of all primary keys are unique, and each entity set participates only once in the relationship. The composition of the primary key for a relationship set depends on the set of attributes associated with the relationship set R.

If the relationship set R has no attributes associated with it, then the set of attributes

$$primary\text{-}key\ (E_i) \cup primary\text{-}key(E_2) \cup \ldots \cup primary\text{-}key(E_n)$$

describes an individual relationship in set R.

If the relationship set R has attributes $a_1, a_2, \ldots a_m$ associated with it, then the set of attributes

$$primary\text{-}key(E_i) \cup primary\text{-}key(E_2) \cup \ldots \cup primary\text{-}key(E_n) \cup \{a_i, a_2, \ldots, a_m\}$$

describes an individual relationship in set R. In both of the above cases, the set of attributes

$$primary\text{-}key(E_i) \cup primary\text{-}key(E_2) \cup \ldots \cup primary\text{-}key(E_n)$$

forms a superkey for the relationship set.

In case the attribute names of primary keys are not unique across entity sets, the attributes are renamed to distinguish them; the name of the entity set combined with the name of the attribute would form a unique name. In case an entity set participates more than once in a relationship set, the role name is used instead of the name of the entity set, to form a unique attribute name.

Overview of Database Design

The database design process can be divided into six steps. The ER model is most relevant to the first three steps:

(1) Requirements Analysis

The very first step in designing a databases application is to understand what data is to be stored in the database, what applications must be built on top of it, and what operations are most frequent and subject to performance requirements. In other words, we must find out what the user wants from the database.

This is usually an informal process that involves discussions with user groups, a study of the current operating environment and how it is expected to change, analysis of any available documentation on existing applications that are expected to be replaced or complemented by the database, and so on. Several methodologies have been proposed for organizing and presenting the information gathered in this step, and some automated tools have been developed to support this process.

(2) Conceptual Database Design

The information gathered in the requirements analysis step is used to develop a high-level description of the data to be stored in the database, along with the constraints that are known to hold over this data. This step is often carried out using the ER model, or a similar high-level data model.

(3) Logical Database Design

We must choose a DBMS to implement our database design, and convert the conceptual database design into a database schema in the data model of the chosen DBMS. We will only consider relational DBMSs and therefore, the track in the logical design step is to convert an ER schema into a relational database schema. The result is a conceptual schema, sometimes called the logical schema, in the relational data model.

Beyond the ER Model

ER modeling is sometimes regarded as a complete approach to designing a logical database schema. This is incorrect because the ER diagram is just an approximate description of the data, constructed through a very subjective evaluation of the information collected during requirements analysis. A more careful analysis can often refine the logical schema obtained at the end of step 3. Once we have a good logical schema, we must consider performance criteria and design the physical schema. Finally, we must address security issues and ensure that users are able to access the data they need, but not data that we wish to hide from them.

(4) Schema Refinement

The fourth step in database design is to analyze the collection of relations in our relational database schema to identify potential problems and to refine it. In contrast to the requirements analysis and conceptual design steps, which are essentially subjective, schema refinement can be guided by some elegant and powerful theory.

(5) Physical Database Design

In this step we must consider typical expected workloads that our database must support and further refine the database design to ensure that it meets desired performance criteria. This step may simply involve building indexes on some tables and clustering some tables, or it may involve a substantial redesign of parts of the database schema obtained from the earlier design steps.

(6) Security Design

In this step, we identify different user groups and different **roles** played by various users (e.g., the development team for a product, the customer support representatives, the product manager). For each role and user group, we must identify the parts of the database that they must be able to access and the parts of the database that they should not be allowed to access, and take steps to ensure that they can access only the necessary parts. A DBMS provides several mechanisms to assist in this step.

In general, our division of the design process into steps should be seen as a classification of the kinds of steps involved in design. Realistically, although we might begin with the six steps process outlined here, a complete database design will probably require a subsequent **tuning** phase in which all six kinds of design steps are interleaved and repeated until the design is satisfactory. Further, we have omitted the important steps of implementing the database design, and designing and implementing the application layers that run on top of the DBMS. In practice, of course, these additional steps can lead to a rethinking of the basic database design.
The concepts and techniques that underlie a relational DBMS are clearly useful to someone who wants to implement or maintain the internals of a database system. However, it is important to recognize that serious users and DBAs must also know how a DBMS works. A good understanding of database system internals is essential for a user who wishes to take full advantage of a DBMS and design a good database.

Type of Entities

1. Existence Dependent

If an entity's existence depends on the existences of one or more other entities it is said to be Existence dependent.

2. Weak Entity

A weak entity is one that :

(i) Is existence dependent; that is, it cannot exist without the entity with which it has the relationship.

(ii) Has a primary key that is partially or totally derived from the parent entity in the relationship.

In other words,

An entity set that does not have sufficient attributes to form a primary key is termed as **weak entity** set. An entity set that has a primary key is termed a **strong entity** set.

The existence of a weak entity is indicated by a double rectangle in an ERD.

For example consider the ERD given below. A company insurance policy may insure an employee and his/her dependents. For the purpose of describing an insurance policy, an EMPLOYEE may or may not have a DEPENDENT, but the DEPENDENT must be associated with an EMPLOYEE. Moreover, the DEPENDENT cannot exist without the EMPLOYEE; that is, you cannot get insurance coverage at the XYZ Corporation as a dependent unless you happen to be a dependent of an employee working for the XYZ Corporation! In other words, DEPENDENT is the weak entity in the relationship "EMPLOYEE has DEPENDENT."

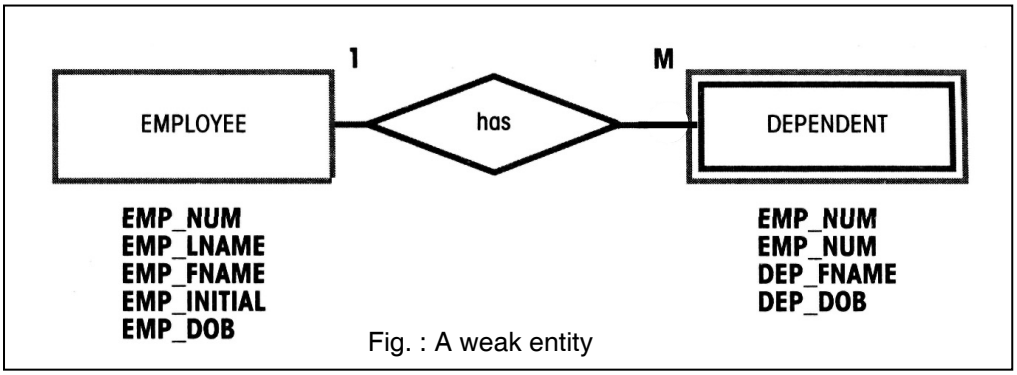


Fig. : A weak entity

As an illustration, consider the entity set *payment*, which has the three attributes: *payment-number, payment-date,* and *payment-amount*. Payment numbers are typically sequential numbers, starting from 1, generated separately for each loan. Thus, although each *payment* entity is distinct, payments for different loans may share the same payment number. Thus, this entity set does not have a primary key; it is a weak entity set.

For a weak entity set to be meaningful, it must be associated with another entity set, called the **identifying** or **owner entity set**. Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be existence dependent on the identifying entity set. The identifying entity set is said to own the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the identifying relationship. The identifying relationship is many to one from the weak entity set to the identifying entity set, and the participation of the weak entity set in the relationship is total.

In our example, the identifying entity set for *payment* is *loan*, and a relationship *loan-payment* that associates *payment* entities with their corresponding *loan* entities is the identifying relationship.

Although a weak entity set does not have a primary key, we nevertheless need a means of distinguishing among all those entities in the weak entity set that depend on one particular strong entity. The discriminator of a weak entity set is a set of attributes that allows this distinction to be made. For example, the discriminator of the weak entity set *payment* is the attribute *payment-number*, since, for each loan, a payment number uniquely identifies one single payment for that loan.The discriminator of a weak entity set is also called the partial key of the entity set.

The primary key of a weak entity set is formed by the primary key of the identifying entity set, plus the weak entity set's discriminator. In the case of the entity set *payment*, its primary key is {*loan-number, payment-number*}, where *loan-number* is the primary key of the identifying entity set, namely *loan*, and *payment-number* distinguishes *payment* entities within the same loan.

In E-R diagrams, a doubly outlined box indicates a weak entity set, and a doubly outlined diamond indicates the corresponding identifying relationship.

In figure given below, the weak entity set *payment* depends on the strong entity set *loan* via the relationship set *loan-payment.* The discriminator of a weak entity set is underlined with a dashed line.

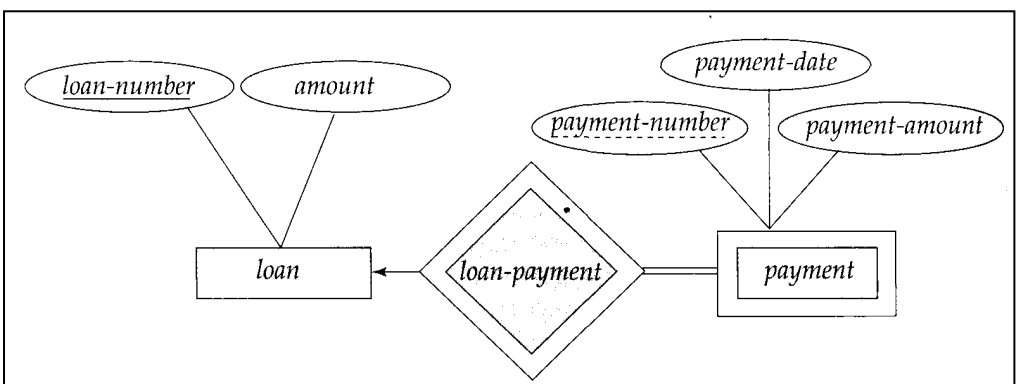


Fig. : E–R diagram with a weak entity set

3. Recursive Entities

A recursive entity is one in which a relationship can exist between occurrences of the same entity set. (Naturally, such a condition is found within a unary relationship.)

For example, a 1:M unary relationship may be expressed by "an EMPLOYEE may manage many EMPLOYEES and each EMPLOYEE is managed by one EMPLOYEE." And, as long as polygamy is not legal, a 1:1 unary relationship may be expressed by "an EMPLOYEE may be married to one and only one other EMPLOYEE." Finally, the M:N unary relationship may be expressed by "a COURSE may be a prerequisite to many other COURSEs and each COURSE may have many other COURSEs as prerequisites." These relationships are shown in figure below.

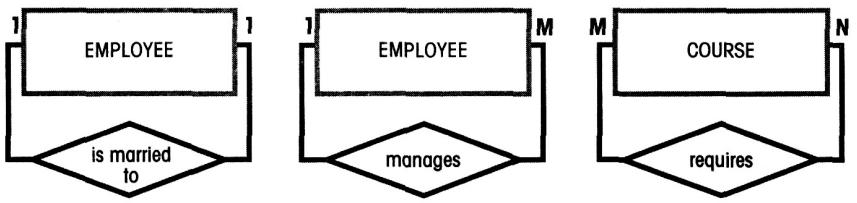


Fig. : Recursive Relationship

4. Composite Entities

If M : N relationships are encountered, we must create a bridge between the entities that display such relationships to convert M : N relationships into 1 : M relationships (Relational model requires the use of 1 : M relationships). The bridge is an entity composed of the primary keys of each of the entities to be connected. The bridge entity is known a **composite entity**.
A composite entity is represented by diamond shape within a rectangular.

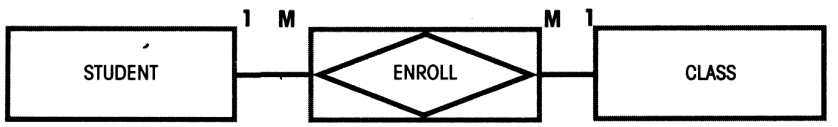


Fig. : Composite Entity

EXTENDED E–R FEATURES

Although the basic E–R concepts can model most database features, some aspects of a database may be more aptly expressed by certain extensions to the basic E–R model.

1. Specialization

An entity set may include subgroupings of entities that are distinct in some way from other entities in the set. For instance, a subset of entities within an entity set may have attributes that are not shared by all the entities in the entity set. The E-R model provides a means for representing these distinctive entity groupings.

Consider an entity set *person,* with attributes *name, street,* and *city.* A person may be further classified as one of the following:

- *customer*
- *employee*

Each of these person types is described by a set of attributes that includes all the attributes of entity set *person* plus possibly additional attributes. For example, *customer* entities may be described further by the attribute *customer-id,* whereas *employee* entities may be described further by the attributes *employee-id* and *salary.* The process of designating subgroupings within an entity set is called **specialization.** The specialization of *person* allows us to distinguish among persons according to whether they are employees or customers.

We can apply specialization repeatedly to refine a design scheme. For instance, bank employees may be further classified as one of the following:

- *officer*
- *teller*
- *secretary*

An entity set may be specialized by more than one distinguishing feature. In our example, the distinguishing feature among employee entities is the job the employee performs. Another, coexistent, specialization could be based on whether the person is a temporary (limited-term) employee or a permanent employee, resulting in the entity sets *temporary-employee* and *permanent-employ*ee. When more than one specialization is formed on an entity set, a particular entity may belong to multiple specializations/For instance, a given employee may be a temporary employee who is a secretary.

In terms of an E-R diagram, specialization is depicted by a *triangle* component labeled ISA, as shown in figure on next page. The label ISA stands for "is a" and represents, for example, that a customer "is a" person. The ISA relationship may also be referred to as a superclass-subclass relationship. Higher and lower-level entity sets are depicted as regular entity sets – that is, as rectangles containing the name of the entity set.

2. Generalization

The refinement from an initial entity set into successive levels of entity subgroupings represents a top-down design process in which distinctions are made explicit. The design process may also proceed in a bottom-up manner, in which multiple entity sets are synthesized into a higher-level entity set on the basis of common features. The database designer may have first identified a *customer* entity set with the attributes *name, street, city,* and *customer-id* and an *employee* entity set with the attributes *name, street, city, employee-id* and *salary.*

There are similarities between the *customer* entity set and the *employee* entity set in the sense that they have several attributes in common. This commonality can be expressed by **generalization**, which is a containment relationship that exists between a *higher-level* entity set and one or more *lower-level* entity sets. In our example, *person* is the higher-level entity set and *customer* and *employee* are lower-level entity sets. Higher and lower-level entity sets also may be designated by the terms superclass and subclass, respectively. The *person* entity set is the superclass of the *customer* and *employee* subclasses.

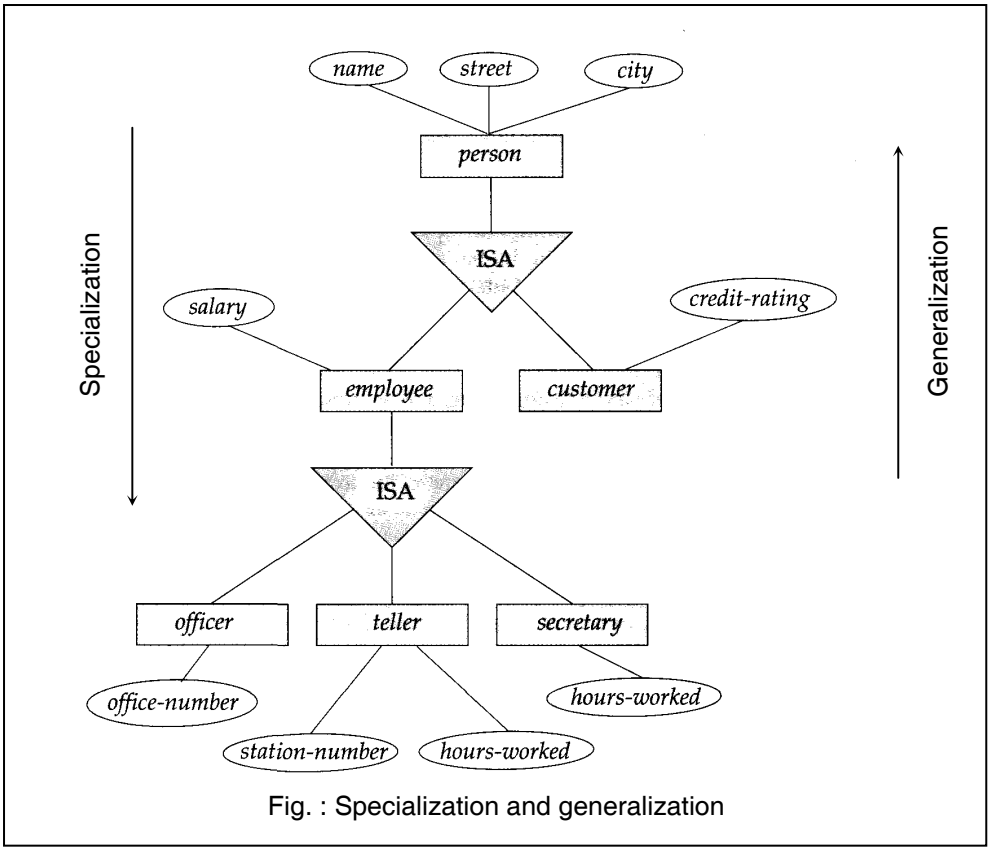


Fig. : Specialization and generalization

3. Attribute Inheritance

A crucial property of the higher and lower-level entities created by specialization and generalization is attribute inheritance. The attributes of the higher-level entity sets are said to be inherited by the lower-level entity sets. For example, *customer* and *employee* inherit the attributes *of person.* Thus, *customer* is described by its *name, street,* and *city* attributes, and additionally a *customer-id* attribute; *employee* is described by its *name, street,* and *city* attributes, and additionally *employee-id* and *salary* attributes.

Whether a given portion of an E-R model was arrived at by specialization or generalization, the outcome is basically the same:

- A higher-level entity set with attributes and relationships that apply to all of its lower-level entity sets
- Lower-level entity sets with distinctive features that apply only within a particular lower-level entity set.

In what follows, although we often refer to only generalization, the properties that we discuss belong fully to both processes.

Figure above, depicts a hierarchy of entity sets. In the figure, *employee* is a lower-level entity set of *person* and a higher-level entity set of the *officer, teller* and *secretary* entity sets. In a hierarchy, a given entity set may be involved as a lower-level entity set in only one ISA relationship; that is, entity sets in this diagram have only single inheritance. If an entity set is a lower-level entity set in more than one ISA relationship, then the entity set has multiple inheritance and the resulting structure is said to be a *lattice.*

4. Aggregation

One limitation of the E-R model is that it cannot express relationships among relationships. To illustrate the need for such a construct, consider the ternary relationship *works-on,* which is shown in figure (a) below, between a *employee, branch,* and *job.* Now, suppose we want to record managers for tasks performed by an employee at a branch; that is, we want to record managers for *(employee, branch, job)* combinations. Let us assume that there is an entity set *manager*

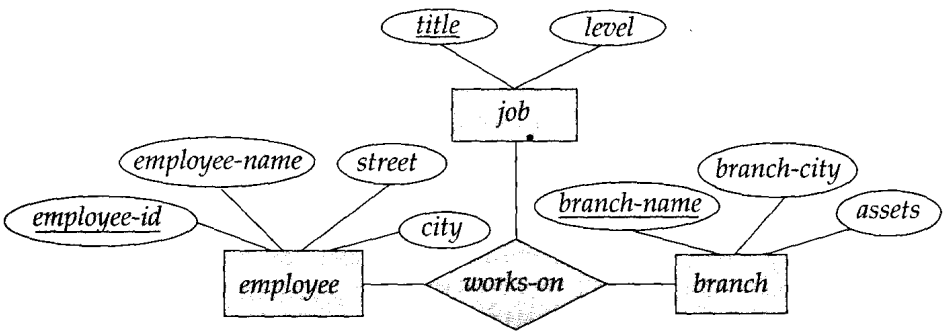


Fig.(a) : E-R diagram with a ternary relationship

One alternative for representing this relationship is to create a quaternary relationship *manages* between *employee, branch, job,* and *manager.* (A quaternary relationship is required -a binary relationship between *manager* and *employee* would not permit us to represent which *(branch, job)* combinations of an employee are managed by which manager.) Using the basic E-R modeling constructs, we obtain the E- R diagram of figure (b) (We have omitted the attributes of the entity sets, for simplicity.)

It appears that the relationship sets *works-on* and *manages* can be combined into one single relationship set. Nevertheless, we should not combine them into a single relationship, since some *employee, branch, job* combinations many not have a manager.

There is redundant information in the resultant figure, however, since every *employee, branch, job* combination in *manages* is also in *works-on.* If the manager were a value rather than a *manager* entity, we could instead make *manager* a multivalued attribute of the relationship *works-on.* But doing so makes it more difficult (logically as well as in execution cost) to find, for example, employee-branch-job triples for which a manager is responsible. Since the manager is a *manager* entity, this alternative is ruled out in any case.

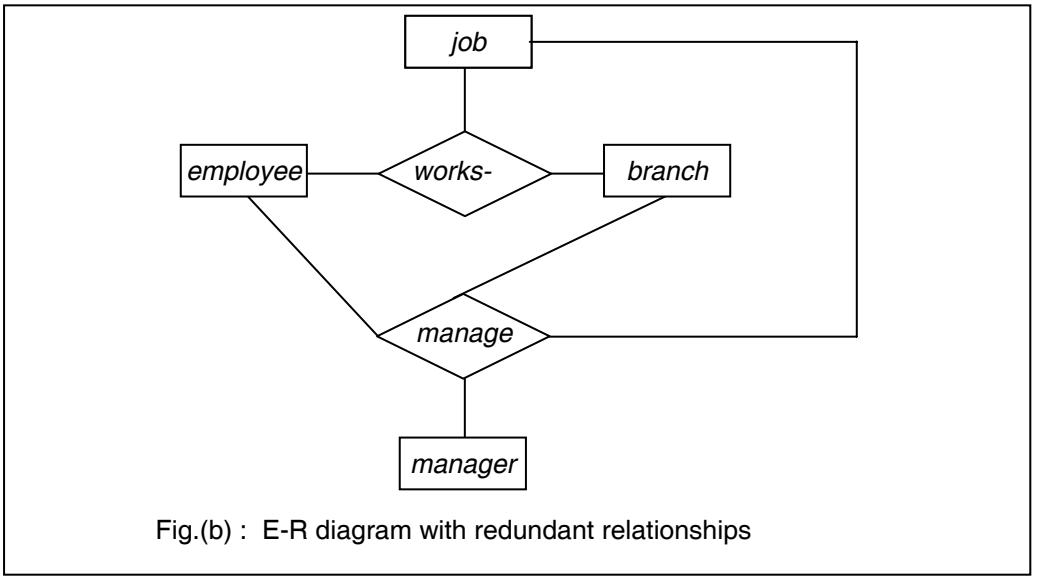


Fig.(b) : E-R diagram with redundant relationships

The best way to model a situation such as the one just described is to use aggregation. **Aggregation** is an abstraction through which relationships are treated as higher-level entities. Thus, for our example, we regard the relationship set *works-on* (relating the entity sets *employee, branch,* and *job)* as a higher-level entity set called *works-on.* Such an entity set is treated in the same manner as is any other entity set. We can then create a binary relationship *manages* between *works-on* and *manager* to represent who manages what tasks. Figure (c) below shows a notation for aggregation commonly used to represent the above situation.

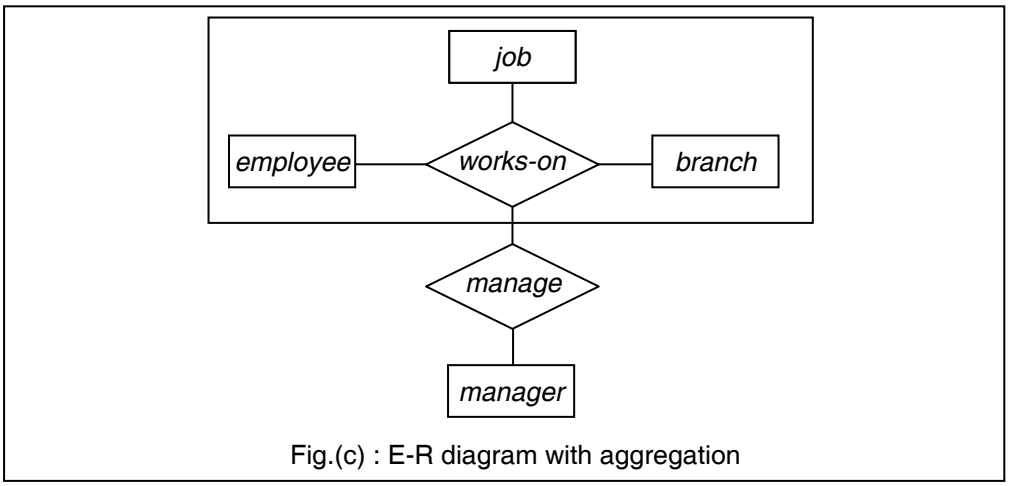


Fig.(c) : E-R diagram with aggregation

Basic Design Issues

The notions of an entity set and a relationship set are not precise, and it is possible to define a set of entities and the relationships among them in a number of different ways. Here we examine basic issues in the design of an E−R database schema.

- Whether to use an attribute or an entity set to represent an object.
- Whether a real–world concept is expressed more accurately by an entity set or by a relationship set.
- Whether to use a ternary relationship or a pair of binary relationship.
- Whether to use a strong or a weak entity set; a strong entity set and its dependent weak entity sets may be regarded as a single "object" in the database, since weak entities are existence dependent on a strong entity.
- Whether using generalization is appropriate; generalization, or a hierarchy of ISA relationships, contributes to modularity by allowing common attributes of similar entity sets to be represented in one place in an E–R diagram.
- Whether using aggregation is appropriate; aggregation groups a part of an E–R diagram into a single entity set, allowing us to treat the aggregate entity set as a single unit without concern for the details of its internal structure.

Symbols used in the E–R notation

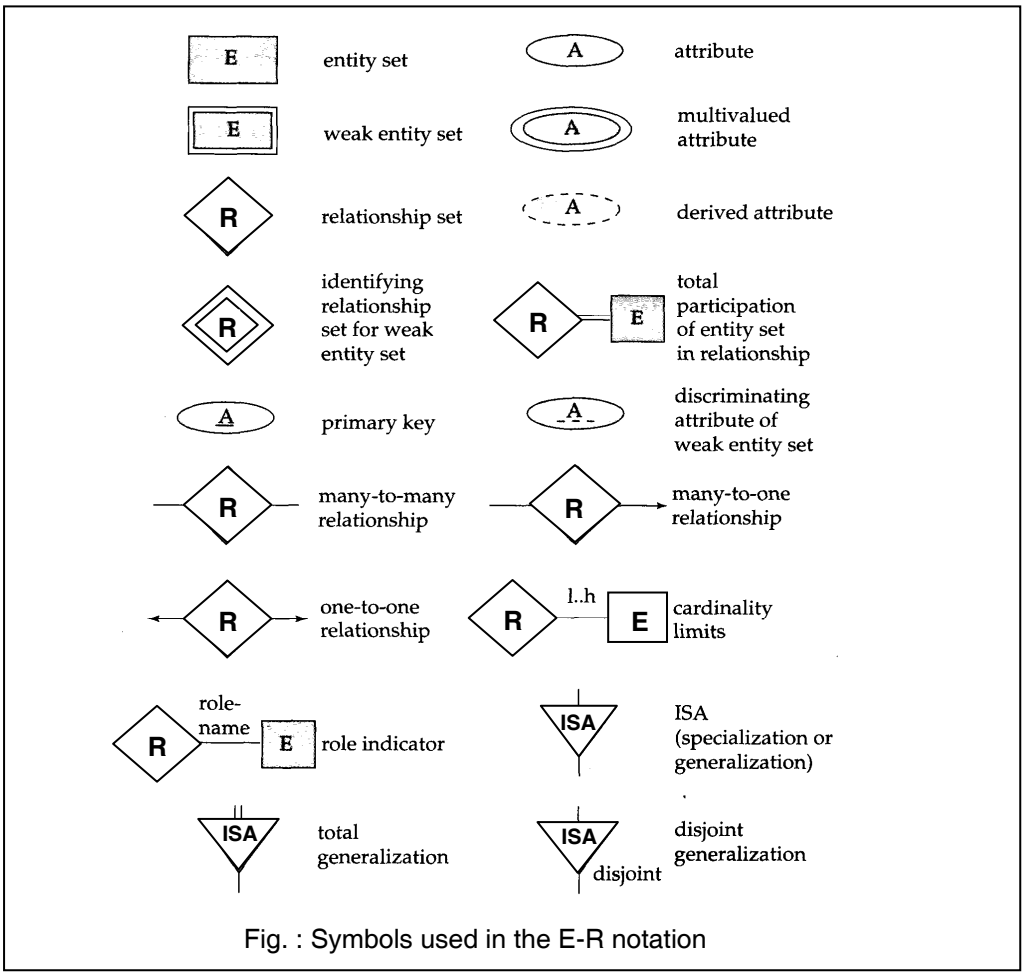


Fig. : Symbols used in the E-R notation

Relational Model

The relational model today is the primary data model for commercial data–processing applications. It has attained its primary position because of its simplicity, which eases the job of the programmer, as compared to earlier data models such as the network model or the hierarchical model.

Structure of Relational Database

A relational database consists of a collection of tables, each of which is assigned a unique name. A row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of table and the mathematical concept of *relation,* from which the relational data model takes its name. In what follows, we introduce the concept of relation.

Consider the *account* table of figure given below. It has three column headers: *account-number, branch-name* and *balance.* Following the terminology of the relational model, we refer to these headers as attributes (as we did for the E-R model). For each attribute, there is a set of permitted values, called the domain of that attribute. For the attribute *branch-name,* for example, the domain is the set of all branch names. Let D_1 denote the set of all account numbers, D_2 the set of all branch names, and D_3 the set of all balances. Any row of *account* must consist of a 3-tuple $(v_1, v_2, v_3,)$, where v_1 is an account number (that is, v_1 *is* in domain D_1), v_2 is a branch name (that is, v_2 is in domain D_2), and v_3 *is* a balance (that is, v_3 *is in* domain D_3). *In* general, *account* will contain only a subset of the set of all possible rows.

Therefore, *account* is a subset of

$$D_1 \times D_2 \times D_3.$$

In general, a table of n attributes must be a subset of

$$D_1 \times D_2 \times \ldots \times D_{n-1} \times D_n$$

Mathematicians define a relation to be a subset of a Cartesian product of a list of domains. This definition corresponds almost exactly with our definition of *table.* The only difference is that we have assigned names to attributes, whereas mathematicians rely on numeric "names," using the integer 1 to denote the attribute whose domain appears first in the list of domains, 2 for the attribute whose domain appears second, and so on. Because tables are essentially relations, we shall use the mathematical terms relation and tuple in place of the terms table and row. A tuple variable is a variable that stands for a tuple; in other words, a tuple variable is a variable whose domain is the set of all tuples.

account-number	branch-name	balance
A-101	Downtown	500
A-102	Perryridge	400
A-201	Brighton	900
A-215	Mianus	700
A-217	Brighton	750
A-222	Redwood	700
A-305	Round Hill	350

Fig. : The *account* relation

Query Languages

A **query language** is a language in which a user requests information from the database. These languages are usually on a level lighter than that of a standard programming language. Query languages can be categorized as either procedural or non–procedural. In a **procedural language**, the user instructs the system to perform a sequence of operations on the database to compute the desired result. In a **nonprocedural language**, the user describes the desired information without giving a specific procedure for obtaining that information.

THE RELATIONAL ALGEBRA

Relational algebra is one of the two formal query languages associated with the relational model.

The degree of relational completeness can be defined by the extend to which relational algebra is supported. Queries in algebra are composed using a collection of operators. A fundamental property is that every operator in the algebra accepts (one or two) relation instances as arguments and returns a relation instance as the result. This property makes it easy to compose operators to form a complex query - a **relational algebra expression.**

The relational algebra is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation as their result. Each relational query describes a step-by-step procedure for computing the desired answer, based on the order in which operators are applied in the query. The procedural nature of the algebra allows us to think of an algebra expression as a recipe, or a plan, for evaluating a query, and relational systems in fact use algebra expressions to represent query evaluation plans.

The fundamental operations in the relational algebra are select, project, union, set difference, Cartesian product, rename, intersection, natural join, division, and assignment. We will define these operations in terms of the fundamental operations.

Fundamental Operations

The select, project, and rename operations are called *unary* operations, because they operate on one relation. The other three operations i.e. set difference, Cartesian product and union operate on pairs of relations and are, therefore, called *binary* operations.

1. The SELECT Operation

SELECT operation yields values for all attributes found in a table. SELECT can be used to list all of the rows values for each attribute, or it can yield selected rows values for each attribute. In other words, SELECT yields a horizontal subset of a table. The effect of a SELECT is shown in figure below.

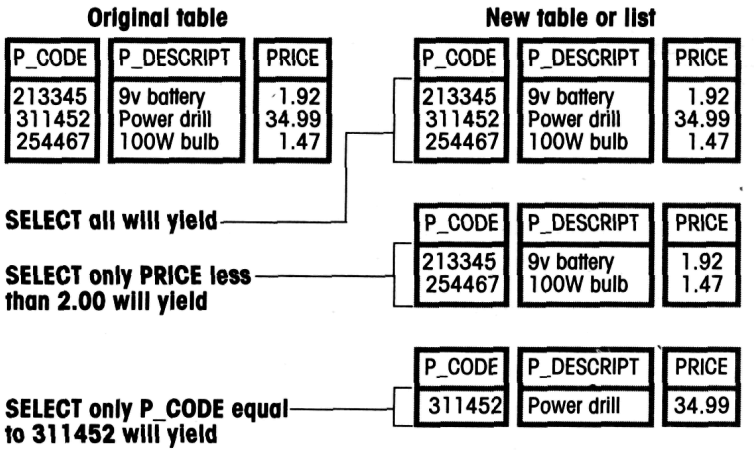


Fig. : The SELECT Operation

The **SELECT** operation selects tuples that satisfy a given predicate. We use the lowercase Greek letter sigma *(σ)* to denote selection. The predicate appears as a subscript to σ.The argument relation is in parentheses after the σ. Thus, to select those tuples of the *Faculty* relation where the department is "*CMPN*", we write

$$\sigma_{dept="CMPN"}(Faculty)$$

If the *Faculty* relation is as shown in figure (a), then the relation that results from the preceding query is as shown in figure (b).

faculty_id	*faculty_name*	*dept*
101	Pankaj	CMPN
102	Dhiraj	SYSTEM
103	Wali	EXTC
104	Bhojewar	IT
105	Nishi	EXTC
106	Sameer	CMPN
107	Pathak	CMPN
108	Ganesh	IT

Fig.(a) : Faculty relation

faculty_id	*faculty_name*	*dept*
101	Pankaj	CMPN
106	Sameer	CMPN
107	Pathak	CMPN

Fig.(b) : Result of $\sigma_{dept="CMPN"}(Faculty)$

We can also find all tuples in which the faculty id is greater than 105 by writing

$$\sigma_{faculty_id>105}(Faculty)$$

In general, we allow comparisons using =, ^, <, ≤, >, ≥ in the selection predicate. Furthermore, we can combine several predicates into a larger predicate by using the connectives *and* (∧), *or* (∨), and *not* (–>). Thus, to find those tuples pertaining to faculties with id greater than 105 and department CMPN, we write

$$\sigma_{dept="CMPN" \wedge faculty_id>105}(Faculty)$$

faculty_id	*faculty_name*	*dept*
106	Sameer	CMPN
107	Pathak	CMPN

Fig. : Result of $\sigma_{dept="CMPN" \wedge faculty_id>105}(Faculty)$

2. The PROJECT Operation

PROJECT produces a list of all values for selected attributes. In other words, PROJECT yields a vertical subset of a table. The effect of a PROJECT is shown in Figure below.

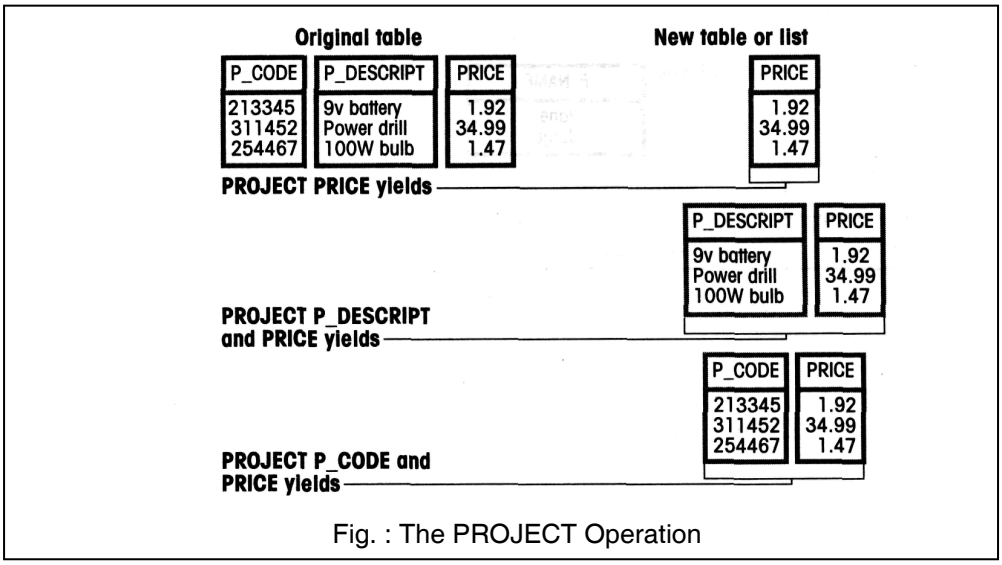


Fig. : The PROJECT Operation

The **PROJECT** operation is a unary operation that returns its argument relation, with certain attributes left out. Since a relation is a set, any duplicate rows are eliminated. Projection is denoted by the uppercase Greek letter pi (Π). We list those attributes that we wish to appear in the result as a subscript to Π. The argument relation follows in parentheses. Thus, we write the query to list all faculty_ids and the depts. of the faculty as

$$\Pi_{faculty_id,dept}(Faculty)$$

Figure below shows the relation that results from this query.

faculty_id	*dept*
101	CMPN
102	SYSTEM
103	EXTC
104	IT
105	EXTC
106	CMPN
107	CMPN
108	IT

Fig. Result of $\Pi_{\text{faculty_id, dept}}(\text{Faculty})$

3. The UNION Operation

UNION combines all rows from two tables. The tables must have the same attribute characteristics (the columns and domains must be identical) to be used in the UNION. When two or more tables share the same columns and domains, they are said to be **union-compatible.** The effect of a UNION is shown in figure below.

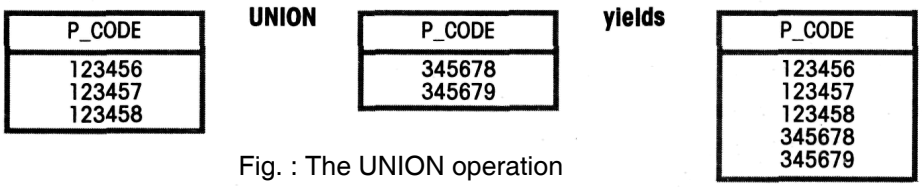


Fig. : The UNION operation

For a union operation r ∪ s to be valid, we require that two conditions hold:

1. The relations *r* and s must be of the same parity. That is, they must have the same number of attributes.
2. The domains of the ith attribute of *r* and the ith attribute of s must be the same, for all *i.*

Note that *r* and s can be, in general, temporary relations that are the result of relational-algebra expressions.

Consider a query to find the faculty ids of faculties who have either an rating of A or a project on Imaging or both. (Refer tables given below)

faculty_id	*age*	*rating*
101	30	C
102	30	B
103	38	A
104	32	C
105	28	B
106	35	A
107	43	A
108	24	D

Fig.: Rating relation

project_id	*faculty_id*	*project_name*
P–08	101	Embedded
P–19	103	Database
P–21	107	Web design
P–26	107	Automation
P–29	103	Robotics
P–33	108	Imaging

Fig.: Project relation

To answer the above query, we need the information in the Rating relation and in the Project relation.
We know how to find the faculty ids with a rating of A:

$$\Pi_{\text{faculty_id}}\left(\sigma_{\text{rating='A'}}(\text{Rating})\right)$$

Note : Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation. In general, since the result of a relational algebra operation is of the same type (relation) as its inputs, relational algebra operations can be composed together into a relational algebra expression.

We also know how to find the faculty ids of faculties having a project on Imaging:

$$\Pi_{\text{faculty_id}}\left(\sigma_{\text{project_name='Imaging'}}(\text{Project})\right)$$

To answer the given query we need the union of above two sets. So the needed expression is,

$$\Pi_{\text{faculty_id}}\left(\sigma_{\text{faculty_id='A'}}(\text{Rating})\right) \cup \Pi_{\text{faculty_id}}\left(\sigma_{\text{project_name='Imaging'}}(\text{Project})\right)$$

The result relation for this query appears as given in table below:

faculty_id
103
106
107
108

Fig. : Result of $\Pi_{\text{faculty_id}}\left(\sigma_{\text{faculty_id='A'}}(\text{Rating})\right) \cup \Pi_{\text{faculty_id}}\left(\sigma_{\text{project_name='Imaging'}}(\text{Project})\right)$

4. The SET DIFFERENCE Operation

SET DIFFERENCE yields all rows in one table that are not found in the other table; that is, it subtracts one table from the other. As was true in the case of UNION, the tables must be union-compatible to yield valid results. The effect of a SET DIFFERENCE is shown in Figure below.

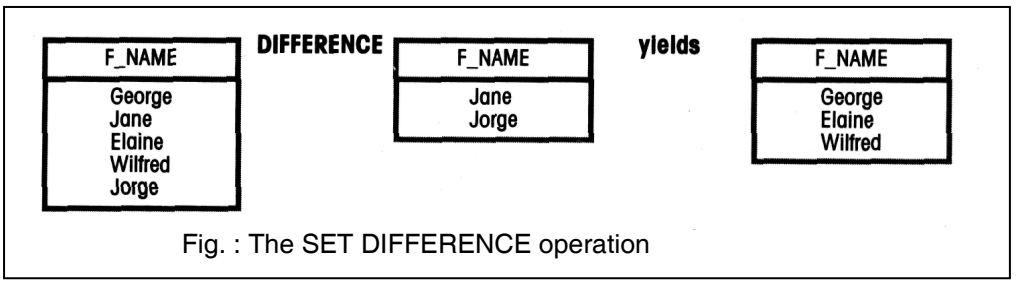


Fig. : The SET DIFFERENCE operation

The **SET-DIFFERENCE** operation, denoted by –, allows us to find tuples that are in one relation but are not in another. The expression r – *s* produces a relation containing those tuples in *r* but not in *s.*

Consider a query to find the faculty ids of faculties having a rating of A but not having a project of Database.

$$\Pi_{\text{faculty_id}}\left(\sigma_{\text{rating='A'}}(\text{Rating})\right) - \Pi_{\text{faculty_id}}\left(\sigma_{\text{project_name='Database'}}(\text{Project})\right)$$

The result relation for this query appears in the figure below :

As with the union operation, we must ensure that set differences are taken between *compatible* relations. Therefore, for a set difference operation r – s to be valid, we require that the relations *r* and s be of the same parity, and that the domains of the ith attribute of *r* and the ith attribute of s be the same.

faculty_id
106
107

Fig. Result of $\Pi_{faculty_id}\left(\sigma_{faculty_id='A'}(Rating)\right) - \Pi_{faculty_id}\left(\sigma_{project_name='Database'}(Project)\right)$

5. The CARTESIAN–PRODUCT Operation

PRODUCT produces a list of all possible pairs of rows from two tables. Therefore, if one table has 10 rows and the other table has 20 rows, the PRODUCT yields a list composed of 10 × 20 = 200 rows. The effect of a PRODUCT is shown in figure below.

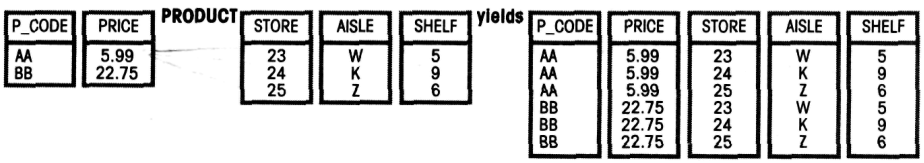


Fig. : The CARTESIAN–PRODUCT Operation

The CARTESIAN–PRODUCT Operation, denoted by a cross (×), allows us to combine in formation from any two relations. We write the cartesian product of relations r_1 and r_2 as

$$r_1 \times r_2$$

It may happen that the same attribute name may appear in both r_1 and r_2. So we need to devise a naming schema to distinguish between these attributes. We do so hereby attaching to an attribute the name of the relation from which the attribute originally came.

For example, the relation schema for Faculty × Rating is,

(*Faculty. faculty_id, faculty_name, dept, Rating. faculty_id, age, rating)*

Consider a query to find the names of all faculties having age 30.

The relational algebra expression for the above query is

$$\Pi_{faculty_name}\left(\sigma_{Faculty.faculty_id\ =\ Rating.faculty_id}\left(\sigma_{age=30}(Faculty \times Rating)\right)\right)$$

Faculty. faculty_id	*faculty_name*	*dept*	*Rating. faculty_id*	*age*	*Rating*
101	Pankaj	CMPN	101	30	C
102	Dhiraj	SYSTEM	101	30	C
103	Wali	EXTC	101	30	C
104	Bhojewar	IT	101	30	C
105	Nishi	EXTC	101	30	C
106	Sameer	CMPN	101	30	C
107	Pathak	CMPN	101	30	C
108	Ganesh	IT	101	30	C
101	Pankaj	CMPN	102	30	B
102	Dhiraj	SYSTEM	102	30	B
103	Wali	EXTC	102	30	B
104	Bhojewar	IT	102	30	B
105	Nishi	EXTC	102	30	B
106	Sameer	CMPN	102	30	B
107	Pathak	CMPN	102	30	B
108	Ganesh	IT	102	30	B
:	:	:			
101	Pankaj	CMPN	107	43	A
102	Dhiraj	SYSTEM	107	43	A
103	Wali	EXTC	107	43	A
104	Bhojewar	IT	107	43	A
105	Nishi	EXTC	107	43	A
106	Sameer	CMPN	107	43	A
107	Pathak	CMPN	107	43	A
108	Ganesh	IT	107	43	A
101	Pankaj	CMPN	108	24	D
102	Dhiraj	SYSTEM	108	24	D
103	Wali	EXTC	108	24	D
104	Bhojewar	IT	108	24	D
105	Nishi	EXTC	108	24	D
106	Sameer	CMPN	108	24	D
107	Pathak	CMPN	108	24	D
108	Ganesh	IT	108	24	D

Fig. : Result of Faculty $\times$ Rating

Faculty. faculty_id	*faculty_name*	*dept*	*Rating. faculty_id*	*age*	*rating*
101	Pankaj	CMPN	101	30	C
102	Dhiraj	SYSTEM	101	30	C
103	Wali	EXTC	101	30	C
104	Bhojewar	IT	101	30	C
105	Nishi	EXTC	101	30	C
106	Sameer	CMPN	101	30	C
107	Pathak	CMPN	101	30	C
108	Ganesh	IT	101	30	C
101	Pankaj	CMPN	102	30	B
102	Dhiraj	SYSTEM	102	30	B
103	Wali	EXTC	102	30	B
104	Bhojewar	IT	102	30	B
105	Nishi	EXTC	102	30	B
106	Sameer	CMPN	102	30	B
107	Pathak	CMPN	102	30	B
108	Ganesh	IT	102	30	B

Fig. : Result of $\sigma_{age=30}(\text{Faculty} \times \text{Rating})$

Faculty. faculty_id	*faculty_name*	*dept*	*Rating. faculty_id*	*age*	*rating*
101	Pankaj	CMPN	101	30	C
102	Dhiraj	SYSTEM	102	30	B

Fig. : Result of $\sigma_{\text{Faculty.faculty_id = Rating.faculty_id}}\left(\sigma_{age=30}(\text{Faculty} \times \text{Rating})\right)$

faculty_name
Pankaj
Dhiraj

Fig : Result of $\prod_{\text{faculty_name}}\left(\sigma_{\text{Faculty.faculty_id = Rating.faculty_id}}\left(\sigma_{age=30}(\text{Faculty} \times \text{Rating})\right)\right)$

6. The RENAME Operation

Unlike relations in the database, the results of relational–algebra expressions do not have a name that we can use to refer to them. It is useful to be able to give them names; the **rename** operator, denoted by the lowercase Greek letter rho(ρ), lets us do this. Given a relational–algebra expression E, the expression

$$\rho_x(E)$$

returns the result of expression E under the name x.

A relation r by itself is considered a (trivial) relational–algebra expression. Thus, we can also apply the rename operation to a relation r to get the same relation under a new name.

A second form of the rename operation is as follows. Assume that a relational–algebra expression e has parity n. Then, the expression

$$\rho_x(A_1, A_2, A_3, ...A_n)(E)$$

returns the result of expression E under the name x, and with the attributes renamed to $A_1, A_2, A_3..., A_n$.

Additional Operators

The fundamental operations of the relational algebra are sufficient to express any relational-algebra query. However, if we restrict ourselves to just the fundamental operations, certain common queries are lengthy to express. Therefore, we define additional operations that do not add any power to the algebra, but simplify common queries. For each new operation, we give an equivalent expression that uses only the fundamental operations.

7. The SET–INTERSECTION Operation

The SET–INTERSECT produces a listing that contains only the rows that appear in both tables. As was true in the case of UNION, the tables must be union-compatible to yield valid results. For example, you cannot use INTERSECT if one of the attributes is numeric and one is character-based. The effect of an INTERSECT is shown in Figure below.

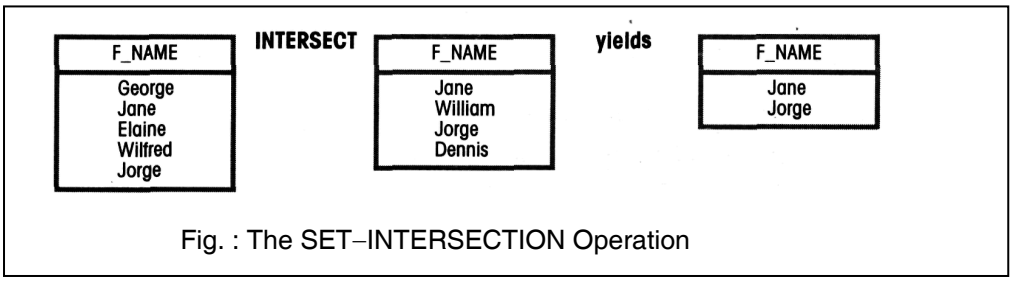


Fig. : The SET–INTERSECTION Operation

We can rewrite any relational algebra expression that uses set intersection by replacing the intersection operation with a pair of set–difference operations as :

$$r \cap s = r - (r - s)$$

Thus, set intersection is not a fundamental operation and does not add any power to the relational algebra. It is simply more convenient to write $r \cap s$ than to write $r - (r - s)$.

Consider a query to find the ids of faculties having a project and rating of A.

The relational algebra expression for the above query is,

$$\prod_{\text{faculty_id}}(\text{Project}) \cap \prod_{\text{faculty_id}}\left(\sigma_{\text{rating=A}}(\text{Rating})\right)$$

The result for the above query is as shown below :

faculty_id
103
107

Fig. : Result of $\prod_{\text{faculty_id}}(\text{Project}) \cap \prod_{\text{faculty_id}}\left(\sigma_{\text{rating=A}}(\text{Rating})\right)$

8. The JOIN Operation

The JOIN allows us to combine information from two or more tables. Join is the real power behind the relational database, allowing the use of independent tables linked by common attributes.

(i) Natural Join

It is often desirable to simplify certain queries that require a Cartesian product. Usually, a query that involves a Cartesian product includes a selection operation on the result of the Cartesian product.

The natural join is a binary operation that allows us to combine certain selections and a Cartesian product into one operation. It is denoted by the "join" symbol $\bowtie$. The natural–join operation forms a Cartesian product of its two arguments, perform a selection forcing equality on those attributes that appear in both relation schemas, and finally removes duplicate attributes.

Although the definition of natural join is complicated, the operation is easy to apply.

Formal definition of the natural join : Consider two relations r(R) and s(S). The natural join of r and s, denoted by r $\bowtie$ s, is a relation on schema R $\cup$ S formally defined as follows :

$$r \bowtie s = \Pi_{R \cup S} \left(\sigma_{r.A_1 = s.A_1 \wedge r.A_2 = s.A_2 \wedge \ldots \wedge r.A_n = s.A_n} r \times s \right)$$

where $R \cap S = \{A_1, A_2, \ldots A_n\}$

As an example the result of Rating $\bowtie$ Project is as given below

faculty_id	*age*	*rating*	*project_id*	*project_name*
101	30	C	P–08	Embedded
103	38	A	P–19	Database
103	38	A	P–29	Robotics
107	43	A	P–26	Automation
107	43	A	P–21	Web design
108	24	D	P–33	Imaging

Fig. : Result of Rating $\bowtie$ Project

Consider the following two relations :

loan_schema = (branch_name, loan numbers, amount)
Borrower_schema = (customer_name, loan number)

Consider the query "Find the names of all customers who have a loan at the bank and find the amount of the loan." We first form the Cartesian product of the borrower and loan relations. Then, we select those tuples that pertain to only the same loan–number, followed by the projection of the resulting customer–name, loan number and amount :

$$\Pi_{\text{customer–name, loan.loan–number, amount}}$$
$$(\sigma_{\text{borrower.loan–number} = \text{loan.loan–number}} (\text{borrower} \times \text{loan}))$$

The natural join is a binary operation that allows us to combine certain selections and a Cartesian product into one operation. It is denoted by the "join" symbol $\bowtie$.

Although the definition of natural join is complicated, the operation is easy to apply. As an illustration, let us consider again the example "Find the names of all customers who have a loan at the bank and find the amount of the loan." This query can be expressed using the natural join as follows :

$$\Pi_{\text{cutstomer-name, loan-number, amount}} (\text{borrower} \bowtie \text{loan})$$

Since the schemas for borrower and loan (that is Borrower–schema and Loan–schema) have the attribute loan–number in common, the natural join operation considers only pairs of tuples that have the same value on loan–number. It combines each such pair of tuples into a single tuple on the union of the two schemas (that is, customer–name, branch–name, loan–number, amount). After performing the projection, we obtain the relation shown below.

Customer–name	loan–number	amount
Jones	L–17	1000
Smith	L–23	2000
Hayes	L–15	1500
Jackson	L–14	1500
Curry	L–93	500
Smith	L–11	900
Williams	L–17	1000
Adams	L–16	1300

Result of $\Pi_{\text{customer-name, loan-number, amount}}$ (borrower $\bowtie$ loan)

(ii) Equijoin

The equijoin operator is a binary operator for combining two relations not necessarily different. In general, this operator combines two relations on all their common attributes. That is, the join consists of all the tuples resulting from concatenating the tuples of the first relation with the tuples of the second relation that have identical values for a common set of attributes X. By common attributes we mean attributes that, although they may not have the same name, but they must have the same domain and underlying meaning. Mathematically, this definition can be expressed as follows : Let r be a relation with a set of attributes R and let s be another relation with a set of attributes S. In addition, let us assume that R and S have some common attributes and let X be that set of common attributes. That is, $R \cap S = X$. The join of r and s, denoted by *r Join s,* is a new relation whose attributes are the elements of $R \cup S$. In addition, for *every tuple* t of the *r Join s* relation, the following three conditions must be satisfied simultaneously :

(1) $t(R) = t_r$ for some tuple t_r of the relation r
(2) $t(S) = t_s$ for some tuple t_s of the relation and
(3) $t_s(X) = t_r(X)$.

An equivalent definition of the Join operator is as follows :

$$r \text{ Join } s = \{\widehat{rs} / s \in r \text{ and } s \in s \text{ and } r (R \cap S) = s(R \cap S)\}$$

(iii) Condition Join (Theta Join)

The most general version of the join operation accepts a join predicate θ and a pair of relation instances as arguments, and returns a relation instance. The theta join operation is an extension to the to the natural join operation that allows us to combine a selection and a Cartesian product into a single operation.
The join condition is identical to a selection condition in form. The operation is defined as follows :

$$R \bowtie_{\theta} S = \sigma_{\theta} (R \times S)$$

Thus $\bowtie$ is defined to be a cross–product followed by a selection. Note that the predicate θ can (and typically does) refer to attributes of both R and S.

(iv) Outer Join

In a Outer join the unmatched pairs would be retained and the values of the unmatched other table would be left blank or null.

The outer-join operation is an extension of the join operation to deal with missing information. Suppose that we have the relations with the following schemas, which contain data on full-time employees:

employee (employee-name, street, city)
ft-works (employee-name, branch-name, salary)

Consider the *employee and ft-works* relations in Figure below.

employee-name	*street*	*city*
Coyote	Toon	Hollywood
Rabbit	Tunnel	Carrotville
Smith	Revolver	Death Valley
Williams	Seaview	Seattle

employee-name	*branch-name*	*salary*
Coyote	Mesa	1500
Rabbit	Mesa	1300
Gates	Redmond	5300
Williams	Redmond	1500

Fig. : The *employee* and *ft–works* relations

Suppose that we want to generate a single relation with all the information (street, city, branch name, and salary) about full-time employees. A possible approach would be to use the natural join operation as follows:

employee $\bowtie$ *ft-works*

The result of this expression appears in figure below.

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500

Fig. : Result of *employee* $\bowtie$ *ft-works*

Notice that we have lost street and city information about Smith, since the tuple describing Smith is absent from *the ft-works* relation; similarly, we have lost the branch name and salary information about Gates, since the tuple describing Gates is absent from the *employee* relation, we can use the *outer-join* operation to avoid this loss of information.

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Smith	Revolver	Death Valley	*null*	*null*
Gates	*null*	*null*	Redmond	5300

Fig. : Result of employee *outer join* ft–works

There are three forms of the operation : left outer join, denoted $\supset\!\bowtie$; right outer join, denoted $\bowtie\!\subset$; and full outer join, denoted $\supset\!\bowtie\!\subset$. All three forms of outer join compute the join, and add extra tuples to the result of the join. The results of the expressions employee $\supset\!\bowtie$ ft–works, employee $\bowtie\!\subset$ ft–works and employee $\supset\!\bowtie\!\subset$ ft–works are shown in figure (a), (b) and (c), respectively.

Employee–name	*Street*	*City*	*Branch–name*	*Salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Smith	Revolver	Death Valley	*null*	*null*

Fig. (a)

The left outer join takes all tuples in the left relation that did not match with any tuple in the right relation, pads the tuples with null values for all other attributes from the right relation, and adds them to the result of the natural join. In Fig.(a), tuple (smith, Revolver, Death Valley, *null*, *null*) is such a tuple. All information from the left relation is present in the result of the left outer join.

The right outer joint is symmetric with the left outer join: Tuples from the right relation that did not match any from the left relation are padded with nulls and added to the result of the natural join. In fig.(b), tuple (Gates, *null*, *null*, Redmond, 5300) is such a tuple. Thus, all information from the right relation is present in the result of the right outer join.

Employee–name	*Street*	*City*	*Branch–name*	*Salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Gates	*null*	*null*	Redmond	5300

Fig. (b)

The full outer join does both of those operations, padding tuples from left relation that did not match any from the right relation, as well as tuples from the right relation that did not match any from the left relation, and adding them to the result of the join. Fig. (c) shows the result of a full outer join.

Employee–name	*Street*	*City*	*Branch–name*	*Salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Smith	Revolver	Death Valley	*null*	*Null*
Gates	*null*	*null*	Redmond	5300

Fig. (c)

Aggregate Functions

Aggregate functions are functions that take a collection of values and return a single value as a result. For example, the aggregate function sum takes a collection of values and returns the **sum** of the values. Thus, the function **sum** applied on the collection returns the value 24. The aggregate

<1, 1, 3, 4, 4, 11>

function **avg** returns the average of the values. When applied to the preceding collection, it returns the value 4. The aggregate function **count** returns the number of the elements in the collection, and would return 6 on the preceding collection. Other common aggregate functions include **min** and **max**, which return the minimum and maximum values in a collection; they return 1 and 11, respectively on the preceding collection.

The collections on which aggregate functions are used can have multiple occurrences of a value ; the order in which the values appear is not relevant. Such collections are called *multisets*. Sets are a special case of multisets, where there is only one copy of each element.

To illustrate the concept of aggregation, we shall use the *pt–works* relation depicted in Fig. (d), showing part–time employees. Suppose that we wanted to find out the total sum of salaries of all the part–time employees in the bank. The relational–algebra expression for this query is :

Employee–name	*Branch–name*	*Salary*
Johnson	Downtown	1500
Loreena	Downtown	1300
Peterson	Downtown	2500
Sato	Austin	1600
Rao	Asustin	1500
Gopal	Perryridge	5300
Adams	Perryridge	1500
Brown	Perryridge	1300

Fig. (d) : The pt_works relation

Sum $_{\text{salary}}$ (pt–works)

The result of this query is a relation with a single attribute, containing a single row with a numerical value corresponding to the sum of all the salaries of all employees working part–time in the bank. If we do want to eliminate duplicates, we use the same function name as before, with the addition of the hyphenated string "**distinct**" appended to the end of the function name (for example, **count–distinct**). An example arises in the query "Find the number of branches appearing in the *pt–works* relation." In this case, a branch name counts only once, regardless of the number of employees working that branch. We write this query as follows.

$$\textbf{count–distinct}_{\text{branch–name}}\ (\text{pt–works})$$

For the relation shown in fig. (d), the result of this query is the value 3.

9. The DIVISION Operation

The DIVISION operation, denoted by ÷, is suited to queries that include the phrase "for all". For the operation of divide operator consider a single column table(Table 1) and a two column table(Table 2) as given below. The result of Table 1 ÷ Table 2 is in Table 3 as shown below.

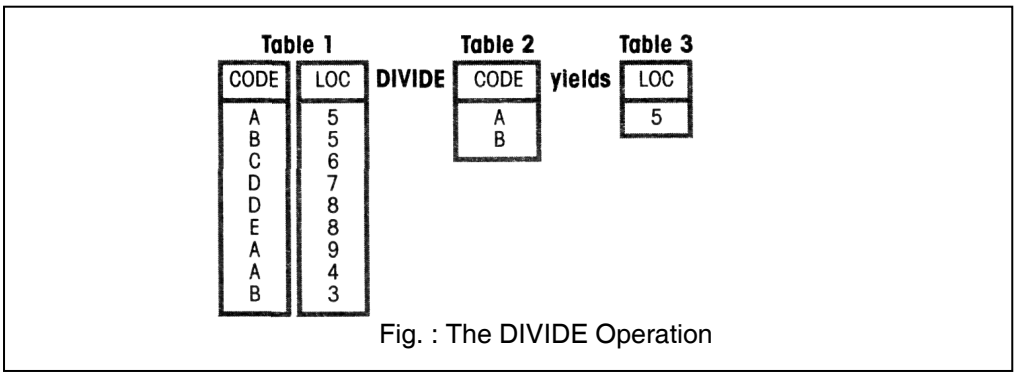


Fig. : The DIVIDE Operation

Note the following points :

a. Table 1 is "divided" by Table 2. Tables 1 and 2 both contain the column CODE but do not share LOC.
b. To be included in the resulting Table 3, a value in the unshared column (LOC) must be associated (in the dividing Table 2) with every value in Table 1.
c. The only value associated with both A and B is 5.

Formally, let *r(R)* and *s(S)* be relations, and let $S \subseteq R$; that is, every attribute of schema *S* is also in schema *R*. The relation $r \div s$ is a relation on schema $R - S$ (that is, on the schema containing all attributes of schema *R.* that are not in schema S). A tuple *t* is in $r \div s$ if and only if both of two conditions hold:

1. *t* is in $\prod_{R-S(r)}$
2. For every tuple t_s in s, there is a tuple t_r in *r* satisfying both of the following :
   a. $t_r[S] = t_s[S]$
   b. $t_r[R - S] = t$

It may surprise you to discover that, given a division operation and the schemas of the relations, we can, in fact, define the division operation in terms of the fundamental operations. Let *r(R)* and *s(S)* be given, with $S \subseteq R$:

$$r \div a = \prod_{R-S}(r) - \prod_{R-S}\left(\left(\prod_{R-S}(r) \times s\right) - \prod_{R-S,S}(r)\right)$$

10. The ASSIGNMENT Operation

It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables. The **assignment** operation, denoted by ←, works like assignment in a programming language. To illustrate this operation, consider the definition of DIVISION. We could write $r \div s$ as

$Temp1 \leftarrow \prod_{R-S}(r)$

$Temp2 \leftarrow \prod_{R-S}((temp1 \times s) - \prod_{R-S,S}(r))$

result = temp1 — temp2

The evaluation of an assignment does not result in any relation being displayed to the user. Rather, the result of the expression to the right of the ← is assigned to the relation variable on the left of the ←. This relation variable may be used in subsequent expressions.

With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query. For relational-algebra queries, assignment must always be made to a temporary relation variable. Assignments to permanent relations constitute a database modification.

Relational Calculus

Relational Calculus is an alternative to relational algebra. In contrast, to the algebra, which is procedural, the calculus is nonprocedural or declarative, in that it allows us to describe the set of answers without being explicit about how they should be computed. Relational calculus has had a big influence on the design of commercial query languages such as SQL and, especially, Query –by – Examples (QBE).

The variant of the calculus that we present in details is called the Tuple Relational Calculus (TRC). Variables in TRC take on tuples as values. In another variant, called the domain relational calculus (DRC), the variables range over field values, TRC has had more of an influence on SQL, while DRC has strongly influence QBE.

THE TUPLE RELATIONAL CALCULUS

When we write a relational – algebra expression, we provide a sequence of procedures that generates the answer to our query. The tuple relational calculus, by contrast, is a nonprocedural query language. It describes the desired information without giving a specific procedure for obtaining that information.

A query in the tuple relational calculus is expressed as;

$\{t \mid P(t)\}$

that is, it is the set of all tuples t such that predicate P is true for t. We use t[A] to denote the value of tuple t on attribute A, and we use $t \in r$ to denote that tuple t is in relation r.

Illustration

Consider the following three relations, *depositor*, *loan*, and *borrower.*

Depositor–schema = (customer–name, account–number)
Loan–schema = (loan–number, branch–name, amount)
Borrower–schema = (customer–name, loan–number)

customer-name	account-number
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Smith	A-215
Turner	A-305

Fig.: The *depositor* relation

loan-number	branch-name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500

Fig.: The *loan* relation

customer-name	loan-number
Adams	L-16
Curry	L-93
Hayes	L-15
Jackson	L-14
Jones	L-17
Smith	L-11
Smith	L-23
Williams	L-17

Fig. : The *borrower* relation

Suppose we want to find the *branch-name, loan-number,* and *amount* for loans of over $1200:

$$\{t \mid t \in loan \wedge t[amount] > 1200\}$$

Suppose that we want only the *loan-number* attribute, rather than all attributes of the *loan* relation. To write this query in the tuple relational calculus, we need to write an expression for a relation on the schema *(loan-number).* We need those tuples on *(loan-number)* such that there is a tuple in *loan* with the *amount* attribute > 1200. To express this request, we need the construct "there exists" from mathematical logic. The notation

$$\exists t \in r(Q(t))$$

means "there exists a tuple *t* in relation *r* such that predicate *Q(t)* is true."

Using this notation, we can write the query "Find the loan number for each loan of an amount greater than $1200" as

$$\{t \mid \exists s \in loan\ (t[loan\text{-}number] = s[loan\text{-}number] \wedge s[amount] > 1200)\}$$

In English, we read the preceding expression as "The set of all tuples *t* such that there exists a tuple s in relation *loan* for which the values of *t* and s for the *loan-number* attribute are equal, and the value of *s* for the *amount* attribute is greater than $1200."

Tuple variable *t* is defined on only the *loan-number* attribute, since that is the only attribute having a condition specified for *t.* Thus, the result is a relation on *(loan-number).*

Consider the query "Find the names of all customers who have a loan from the Perryridge branch." This query is slightly more complex than the previous queries, since it involves two relations: *borrower* and *loan.* As we shall see, however, all it requires is that we have two "there exists" clauses in our tuple-relational-calculus expression, connected by *and* ($\wedge$).

We write the query as follows:

$$\{t \mid \exists s \in \text{ borrower } (t[\text{customer-name}] = s[\text{customer-name}] \wedge \exists u \in \text{ loan } (u[\text{loan-number}] = s[\text{loan-number}] \wedge u[\text{branch-name}] = \text{"Perryridge"}))\}$$

In English, this expression is "The set of all (customer-name) tuples for which the customer has a loan that is at the Perryridge branch." Tuple variable u ensures that the customer is a borrower at the Perryridge branch. Tuple variable s is restricted to pertain to the same loan number as s. Figure below shows the result of this query.

customer-
Adams
Hayes

Fig. : Names of all customers who have a loan at the Perryridge branch.

To find all customers who have a loan, an account, or both at the bank, we used the union operation in the relational algebra. In the tuple relational calculus, we shall need two "there exists" clauses, connected by or ($\vee$):

$$\{t \mid \exists s \in \text{ borrower } (t[\text{customer-name}] = s[\text{customer-name}]) \vee u \in \text{ depositor } (t[\text{customer-name} = u[\text{customer-name}])\}$$

This expression gives us the set of all customer-name tuples for which at least one of the following holds:

- The customer-name appears in some tuple of the borrower relation as a borrower from the bank.
- The customer-name appears in some tuple of the depositor relation as a depositor of the bank.

If some customer has both a loan and an account at the bank, that customer appears only once in the result, because the mathematical definition of a set does not allow duplicate members. The result of this query appeared earlier in Figure below.

customer-name
Adams
Curry
Hayes
Jackson
Jones
Smith
Williams
Lindsay
Johnson
Turner

Fig. : Name of all customers who have either a loan or an account

If we now want only those customers who have both an account and a loan at the bank, all we need to do is to change the or ($\vee$) to and ($\wedge$) in the preceding expression.

$$\{t \mid \exists\ s \in \text{borrower}\ (t[\text{customer-name}] = s[\text{customer-name}])$$
$$\wedge\ u \in \text{depositor}\ (t\{\text{customer-name}] = u[\text{customer-name}])\}$$

The result of this query appeared in Figure below.

customer-
Hayes
Jones
Smith

Fig. **:** Customers with both an account and a loan at the bank

Now consider the query "Find all customers who have an account at the bank but do not have a loan from the bank." The tuple-relational-calculus expression for this query is similar to the expressions that we have just seen, except for the use of the not ($\neg$) symbol:

$$\{t \mid \exists\ u \in \text{depositor}\ (t[\text{customer-name}] = u[\text{customer-name}])$$
$$\wedge \neg \exists\ s \in \text{borrower}\ (t[\text{customer-name}] = s[\text{customer-name}])\}$$

This tuple-relational-calculus expression uses the $\exists$ *u* $\in$ *depositor* (...) clause to require that the customer have an account at the bank, and it uses the $\neg \exists$ s $\in$ *borrower (...*) clause to eliminate those customers who appear in some tuple of the *borrower* relation as having a loan from the bank. The result of this query appeared in Figure below.

customer-
Johnson
Lindsay
Turner

Fig. : Customers with an account but no loan

Formal Definition

We are now ready for a formal definition. A tuple–relational–calculus expression is of the form

$$\{t \mid P(t)\}$$

where P is a formula. Several tuple variables may appear in a formula. A tuple variable is said to be a free variable unless it is quantified by a $\exists$ or $\forall$. Thus, in

$$t \in \text{loan} \wedge \exists\ s \in \text{customer}\ (t\ \{\text{branch–name}] = s\ [\text{branch–name}])$$

t is a free variable. Tuple variable s is said to be a bound variable.

A tuple–relational–calculus formula is built up out of atoms. An atom has one of the following forms :

- s $\in$ r, where s is a tuple variable and r is a relation (we do not allow use of the $\notin$ operator)

- s[x] Θ u[y], where s and u tuple variables, x is an attribute on which s is defined, y is an attribute on which u is defined, and is a comparison operator (<, ≤, =, ≠, >, ≥); we require that attributes x and y have domains whose members can be compared by Θ.

- s[x] Θ c, where s is a tuple variable, x is an attribute on which s is defined, Θ is a comparison operator, and c is a constant in the domain of attribute x.

We build up formulae from atoms by using the following rules :

- An atom is a formula
- If P_1 is a formula, then so are $\neg P_1$ and (P_1).
- If P_1 and P_2 are formulae, then so are $P_1 \vee P_2$, $P_1 \wedge P_2$, and $P_1 \Rightarrow P_2$.
- If P_1 (s) is a formula containing a free tuple variable s, and r is a relation, then
  $\exists$ s $\in$ r (P_1 (s)) and $\forall$ s $\in$ r (P_1(s)) are also formulae.

As we could for the relational algebra, we can write equivalent expressions that are not identical in appearance. In the tuple relational calculus, these equivalences include the following three rules.

1. $P_1 \wedge P_2$ is equivalent to $\neg(\neg(P_1) \vee \neg (P_2))$
2. $\forall$ t $\in$ r (P_1(t)) is equivalent to $\neg \exists$ t $\in$ r($\neg P_1$(t))
3. $P_1 \Rightarrow P_2$ is equivalent to $\neg (P_1) \vee P_2$.

Example of TRC Queries

Consider the relations Faculty and Allotment of a Engineering College.

faculty_id	*name*	*age*	*rating*
101	Pankaj	30	5
102	Dhiraj	30	7
103	Wali	38	10
104	Bhojewar	32	6
105	Nishi	28	7
106	Sameer	35	4
107	Pathak	43	3
108	Ganesh	24	6

Fig. : The Faculty relation

faculty_id	*room_no*	*date*
101	C–101	01/09/03
102	C–104	05/09/03
101	C–105	05/09/03
104	C–101	10/08/03
105	C–103	12/10/03
105	C–107	08/08/03
107	C–110	11/09/03
108	C–104	21/08/03

Fig. : The Allotment relation

1. Find the names and ages of all faculties with a rating above 5.

Soln.

$\{t \mid \exists f \in \text{Faculty } (f.rating > 5 \wedge t.name = f.name \wedge t.age = f.age)\}$

Explanation :

t is consider to a tuple variable with exactly two fields, which are name and age, because these are the only field of t that are mentioned and t does not range over any of the relations in the query; i.e. there is no sub formula of the form $t \in$ Relname. The result of the query is a relation with two fields, name and age. The atomic formulae of t.name = f.name and t.age = f.age gives values to the fields of an answer tuple t.

The answer is a set of tuples as given below :

faculty_id	*name*	*age*	*rating*
102	Dhiraj	30	7
103	Wali	38	10
104	Bhojewar	32	6
105	Nishi	28	7
108	Ganesh	24	6

2. Find the name of Faculty who are taking lectures at Room No. C−101

Soln.

$\{t \mid \exists f \in \text{Faculty } \exists a \in \text{Allotment } (a.faculty_id = f.faculty_id$
$\wedge a.room_no = \text{'C−101'} \wedge t.name = f.name)\}$

Explanation :

This query can be read as follows :

Retrieve all Faculty tuples for which there exist a tuple in Allotment, having the same value in the faculty_id field and with room_no = C−101. The answer tuple t contains just one field name.

The answer is a set of tuples as given below :

name
Pankaj
Bhojewar

3. Find the faculty name, room no, allotment date, for each allotment.

Soln.

$\{t \mid \exists a \in \text{Allotment } \exists f \in \text{Faculty } (a.faculty_id = f.faculty_id$
$\wedge t.room_no = a.room_no$
$\wedge t.date = a.date$
$\wedge t.name = f.name)\}$

Explanation :

For each Allotment tuple, we look for a tuple in Faculty with the same faculty_id. Given a pair of such tuples, we construct and answer tuple t with fields name, room no., allotment date by coping the corresponding fields from these two tuples.

The answer is a set of tuples as given below :

name	*faculty_id*	*date*
Pankaj	101	01/09/03
Dhiraj	102	05/09/03
Pankaj	101	05/09/03
Bhojewar	104	10/08/03
Nishi	105	12/10/03
Nishi	105	08/08/03
Pathak	107	11/09/03
Ganesh	108	21/08/03

LMR (LAST MINUTE REVISION)

Information is derived from data, which are usually stored in a database. To implement a database and to manage its contents you need commercial software known as a database management system (DBMS).

- A database–management system (DBMS) consists of a collection of interrelated data and a collection of programs to access that data. The data describes one particular enterprise.

- The primary goal of a DBMS is to provide an environment that is both convenient and efficient for people to use in retrieving and storing information.

- Database systems are ubiquitous today, and most people interact, either directly or indirectly, with databases many times every day.

- Database systems are designed to store large bodies of information. The management of data involves both the definition of structures for the storage of information and the provision of mechanisms for the manipulation of information. In addition, the database system must provide for the safety of the information stored, in the face of system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.

- A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.

- Underlying the structure of a database is the data model: a collection of conceptual tools for describing data, data relationships, data semantics, and data constraints. The entity-relationship (E-R) data model is a widely used data model, and it provides a convenient graphical representation to view data, relationships and constraints. The relational data model is widely used to store data in databases. Other data models are the object-oriented model, the object-relational model, and semi–structured data models.

- The overall design of the database is called the database schema. A database schema is specified by a set of definitions that are expressed using a data-definition language (DDL).

- A data-manipulation language (DML) is a language that enables users to access or manipulate data. Nonprocedural DMLs, which require a user to specify only what data are needed, without specifying exactly how to get those data, are widely used today.

- Database users can be categorized into several classes, and each class of users usually uses a different type of interface to the database.

- A database system has several subsystems. The transaction manager subsystem is responsible for ensuring that the database remains in a consistent (correct) state despite system failures. The transaction manager also ensures that concurrent transaction executions proceed without conflicting. The query processor subsystem compiles and executes DDL and DML statements. The storage manager subsystem provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- Database applications are typically broken up into a front-end part that runs at client machines and a part that runs at the back end. In two-tier architectures, the front-end directly communicates with a database running at the back end. In three-tier architectures, the back end part is itself broken up into an application server and a database server.

Entity–Relational Model

- The entity-relationship (E-R) data model is based on a perception of a real world that consists of a set of basic objects called entities, and of relationships among these objects.
- An entity is an object that exists in the real world and is distinguishable from other objects. We express the distinction by associating with each entity a set of attributes that describes the object.
- A relationship is an association among several entities. The collection of all entities of the same type is an entity set, and the collection of all relationships of the same type is a relationship set.
- Mapping cardinalities express the number of entities to which another entity can be associated via a relationship set.

- A superkey of an entity set is a set of one or more attributes that, taken collectively, allows us to identify uniquely an entity in the entity set. We choose a minimal superkey for each entity set from among its superkeys; *the minimal superkey is termed the entity set's primary key*. Similarly, a relationship set is a set of one or more attributes that, taken collectively, allows us to identify uniquely a relationship in the relationship set. Likewise, we choose a minimal superkey for each relationship set from among its superkeys; this is the relationship set's primary key.

- An entity set that does not have sufficient attributes to form a primary key is termed a weak entity set. An entity set that has a primary key is termed a strong entity set.

- Specialization and generalization define a containment relationship between a higher-level entity set and one or more lower-level entity sets. Specialization is the result of taking a subset of a higher-level entity set to form a lower-level entity set. Generalization is the result of taking the union of two or more disjoint (lower-level) entity sets to produce a higher-level entity set. The attributes of higher-level entity sets are inherited by lower-level entity sets.

- Aggregation is an abstraction in which relationship sets (along with their associated entity sets) are treated as higher-level entity sets, and can participate in relationships.

E–R Diagram

- Entity relationship (E-R) model uses E-R diagrams to represent the conceptual database as viewed by the end user. The E-R model's three main components are entities, relationships, and attributes. The E-R diagram also includes connectivity and cardinality notations.

- E-R symbols are used to graphically depict the E-R model's components and relationships. For instance, a diamond depicts a relationship, a rectangle depicts an entity, an ellipse depicts an attribute, and a double-hulled rectangle depicts a weak entity. Single attributes are connected to their entities with single lines, and multivalued attributes are connected to their entities with double lines. A derived attribute is connected to its entity with a dashed line. Key attributes are underlined. Optional relationships are indicated by writing a circle (o) next to the optional entity.)

Relational Algebra

- The relational Algebra defines a set of algebraic operation that operate on tables, and output tables as their results. These operations can be combined to get expressions that express desired queries. The algebra defines the basic operations used within relational query languages.

Operations

	Operation	Symbol	Descriptions
1.	SELECT	σ	SELECT operation selects tuples that satisfy a given predicate.
2.	PROJECT	Π	PROJECT operation is a unary operation that returns its argument relation, with certain attributes left out.
3.	UNION	$\cup$	UNION combines all rows from two tables(argument relation)
4.	SET DIFFERENCE	$-$	SET DIFFERENCE yields tuples that are in one relation but are not in another.
5.	CARTESIAN–PRODUCT	$\times$	CARTESIAN–PRODUCT produces a list of all possible pairs of rows from two tables.
6.	RENAME	ρ	RENAME gives name to the result of relational algebra expressions so that we can refer to them.
7.	SET–INTERSECTION	$\cap$	SET–INTERSECTION produces a listing that contains only the row that appear in both tables.
8.	JOIN	$\bowtie$	JOIN allows us to combine information from two or more tables.
9.	DIVISION	$\div$	DIVISION produces the values from the unshared columns of the first argument associated with every value of common columns of the second argument.
10.	ASSIGNMENT	$\leftarrow$	ASSIGNEMENT is used to store results of Relational Algebraic expressions in temporary relation variables.

Tuple Calculus

The tuple relational calculus and the domain relational calculus are non–procedural languages that represent the basic power required in a relational query language. The basic relational algebra is a procedural language that is equivalent in power to both forms of the relational calculus when they are restricted to safe expressions.

❑ ❑ ❑ ❑ ❑ ❑

ASSIGNMENT – 1

Duration : 45 Min. **Max. Marks : 30**

Q1 to Q6 carry one mark each

1. A customer's telephone number is stored in a customer file, a sales agent file, and an invoice file. In database, this is known as _____

(A) Integrity (B) Redundancy
(C) Availability (D) Transparency

2. Find the odd man out :
Desktop database, work group data base, distributed database, enterprise data base.

(A) Desktop database (B) Workgroup database
(C) Distributed database (D) Enterprise database

3. To be considered minimally relational, the DBMS must support the following key relational functions :

(A) SELECT, PROJECT, UNION
(B) SELECT, PROJECT, JOIN
(C) SELECT, UNION, JOIN
(D) SELECT,UNION, INTERSECT

4. The schema of a table is an example of _____

(A) Data dictionary (B) Metadata
(C) Entity (D) Relationship

5. A transaction is a collection of operations that performs a single logical function in a database application. The successful execution of a database despite the possibility of system failure is called _____.

(A) Atomicity (B) Consistency
(C) Durability (D) Integrity

6. Which of the following is (are) non–procedural query languages ?

(i) Relational algebra
(ii) Tuple Relational calculus
(iii) Domain Relational Calculus

(A) (i) only (B) (ii) only
(C) (i) and (ii) only (D) (ii) and (iii) only.

Q7 to Q18 carry two marks each

7. In context with Relational Algebra, which of the following are unary operations.

(i) SELECT (ii) PROJECT
(iii) UNION (iv) PRODUCT

(A) (i) and (iii) only (B) (ii) and (iv) only
(C) (i) and (ii) only (D) All are binary operations

8. Match the following :

1.	SELECT	a)	Vertical subset of a table
2.	PROJECT	b)	Horizontal subset of a table
		c)	Common tuples

(A) 1 – a, 2 – b (B) 1 – b, 2 – a
(C) 1 – c, 2 – a (D) 1 – c, 2 – b

Direction for Q9 to Q12

Answer the questions with reference to entities EMPLOYEE and PROJECT and the relationship ASSIGNED_TO between them.

Emp#	Name
101	Manish
103	Smita
104	Rock
106	Anita
107	Dipti
110	Deepak
112	Priya

Fig. : EMPLOYEE Table

Project#	Emp#
NET453	101
NET354	103
NET343	104
NET354	104
NET231	106
NET278	106
NET353	106
NET354	106
NET453	106
NET231	107
NET353	107
NET278	110
NET353	112
NET354	112

Fig. : ASSIGNED_TO Table

Project#	Project_Name	Project_Leader
NET231	Pascal	107
NET278	Pascal/Object	110
NET353	Database	107
NET354	Operating Sys	104
NET453	Database	101

Fig. : PROJECT Table

9. Write a relational algebra expression to satisfy the following query.

"Get Emp# of employees working on project NET353"

(A) $\pi_{Emp\#}\left(\sigma_{Project\#='NET353'}\left(ASSIGNED_TO\right)\right)$

(B) $\sigma_{Emp\#}\left(\pi_{Project\#='NET353'}\left(ASSIGNED_TO\right)\right)$

(C) $\pi_{Emp\#}\left(\sigma_{Project\#='NET353'}\left(EMPLOYEE \bowtie PROJECT\right)\right)$

(D) None of these

10. Choose the correct response relation from the given option for the following query (Relational Algebra Expression).

$$\text{EMPLOYEE} \bowtie \pi_{\text{Emp\#}}\left(\sigma_{\text{Project\#='NET353'}}(\text{ASSIGNED_TO})\right)$$

(A)

Emp#	Project#
106	NET353
107	NET353
112	NET353

(B)

Emp#
106
107
112

(C)

Emp#	Name
106	Anita
107	Dipti
112	Priya

(D)

Emp#	Project#	Name
106	NET353	Anita
107	NET353	Dipti
112	NET353	Priya

11. Choose the most appropriate relational algebra expression to satisfy the following query.
"Obtain details of employees working on the database project"

(A) $\text{EMPLOYEE} \bowtie \pi_{\text{Emp\#}}\left(\text{PROJECT} \bowtie \left(\pi_{\text{Project\#}}\left(\sigma_{\text{Project_Name='Database'}}(\text{ASSIGNED_TO})\right)\right)\right)$

(B) $\text{EMPLOYEE} \bowtie \pi_{\text{Emp\#}}\left(\text{PROJECT} \bowtie \left(\sigma_{\text{Project\#}}\left(\pi_{\text{Project_Name='Database'}}(\text{ASSIGNED_TO})\right)\right)\right)$

(C) $\text{EMPLOYEE} \bowtie \pi_{\text{Emp\#}}\left(\text{ASSIGNED_TO} \bowtie \left(\pi_{\text{Project\#}}\left(\sigma_{\text{Project_Name='Database'}}(\text{PROJECT})\right)\right)\right)$

(D) $\text{EMPLOYEE} \bowtie \pi_{\text{Emp\#}}\left(\text{ASSIGNED_TO} \bowtie \left(\sigma_{\text{Project\#}}\left(\pi_{\text{Project_Name='Database'}}(\text{PROJECT})\right)\right)\right)$

12. Choose the correct response relation from the given option for the following query .
"Get the number of employees who work on all projects".

(A)

Emp#
107

(B)

Emp#
106

(C)

Emp#
112

(D)

Emp#
101
106

Consider the given schemes (13 – 15).

Branch_scheme = (Branch_name, assets, Branch_city)

Customer_scheme = (customer_name, street, customer_city)

Deposit_scheme = (Branch_name, account_number, Customer_name, balance)

Borrow_scheme = (Branch_name, loan_number, Customer_name, amount)

Client_scheme = (Customer_name, banker_name)

13. Using relational algebra, the query that finds customers, who have a balance of over 1000 is

(A) $\pi_{customer_name}\ (\sigma_{balance>1000}(Deposit))$

(B) $\sigma_{customer_name}(\sigma_{balance>1000}(Deposit))$

(C) $\pi_{customer_name}\ (\sigma_{balance>1000}(Borrow))$

(D) $\sigma_{customer_name}\ (\pi_{balance>1000}(Borrow))$

14. Which of the following queries finds the clients of banker Agassi and the city they live in?

(A) $\pi_{client.customer_name,customer_city}\ (\sigma_{client.customer_name})$

$= customer.customer_name\ (\sigma_{Banker_name = \text{“Agassi”}}\ (client \times customer))$

(B) $\pi_{client.customer_name,,customer_city}\ (\sigma_{Banker_name = \text{“Agassi”}}\ (client \times customer))$

(C) $\pi_{client.customer_name,customer_city}\ (\sigma_{Banker_name = \text{“Agassi”}}(\sigma_{client\ .\ customer_name= customer\ .\ customer_name}\ (client \times customer)))$

(D) $\pi_{customer_name,,customer_city}\ (\sigma_{Banker_name = \text{“Agassi”}}\ (client \times customer))$

15. Which of the following tuple relational calculus finds all the customers who have a loan amount more than 1200 ?

(A) $\{t(Customer_name) / t \in borrow \wedge t[amount] > 1200\}$

(B) $\{t \mid t(Customer_name) \in borrow \wedge t[amount] > 1200\}$

(C) $\{t \mid \exists s \in borrow\ (t[Customer_name]= s[Customer_name] \wedge s[amount]>1200)\}$

(D) $\{t \mid \exists s \in borrow\ (t[Customer_name]= s[Customer_name]) \vee s[amount]>1200)\}$

16. Consider an entity set EMPLOYEE that records information about all the employees of a bank. In this case, EMPLOYEE has a relation is married to another employee of the bank. Such a entity set is called _____.

(A) Binary entity set (B) Cyclic entity set

(C) Recursive entity set (D) Multivalued entity set

17. Match the following :

1.	Composite attribute	a.	Set of more than one attributes
2.	Single–valued attribute	b.	An attributes that cannot be further subdivided
		c.	An attribute than can be further sub–divided to yield additional attributes
		d.	An attribute than can have only a single value.

(A) 1 – a, 2 – d
(B) 1 – a, 2 – b
(C) 1 – c, 2 – d
(D) 1 – c, 2 – b

18. Match the following :

1.	$P_1 \wedge P_2$	a.	$\neg(\neg(P_1) \vee \neg (P_2))$
2.	$P_1 \Rightarrow P_2$	b.	$\neg \exists \in r(\neg P_1(t))$
		c.	$\neg(P_1) \vee P_2$

(A) 1 – a, 2 – b
(B) 1 – a, 2 – c
(C) 1 – c, 2 – a
(D) 1 – c, 2 – b

❑ ❑ ❑ ❑ ❑ ❑

TEST PAPER – 1

Duration : 30 Min. **Max. Marks : 25**

Q1 to Q5 carry one mark each

1. Database that are designed and managed specifically to meet information needs are called _____.

(A) Production databases
(B) Transaction databases
(C) Data Warehouses
(D) Database Management Systems

2. The data retrieval time factor is most critical for _____

(A) Transactional DBMS (B) Decision DBMS
(C) Data Warehouse (D) Both (B) and (C)

3. Which of the following statements is false in context with Relational Database Model

(A) Controlled redundancy makes the relational database work
(B) The attribute B is functionally dependent on the attribute A if each value in column B determines one and only one value in column A.
(C) Within a table, if there exist a key that will uniquely identify each row, then the stable is said to exhibit entity integrity.
(D) If the foreign key contains either matching values or nulls, the table(s) that make use of such a foreign key are said to exhibit referential integrity.

4. Which of the following query transformation (i.e. replacing the l.h.s expression by the r.h.s. expression) is incorrect ? R_1 and R_2 are relations, C_1, C_2 are selection conditions and A_1 and A_2 are attributes of R_1.

(A) $\sigma_{C_2}(R_1 \cup R_2) = \sigma_{C_2}(R_1) \cup \sigma_{C_2}(R_2)$
(B) $\sigma_{C_2}(R_1 \cap R_2) = \sigma_{C_2}(R_1 - (R_1 - R_2))$
(C) $\pi_{A_1}(\sigma_{C_1}(R_1)) = \sigma_{C_1}(\pi_{A_1}(R_1))$
(D) Both (B) and (C)

5. Which of the following is not supported by the hierarchical database model ?

(A) Structural independence (B) Data independence
(C) Database integrity (D) Database security

Q6 to Q15 carry two marks each

6. A file system is said to exhibit data dependence because _____.

(A) A change in any file's structure, such as the addition or deletion of a field, requires the modification of all programs using that file.

(B) Organization of the data, within the file is determined by the data's expected use.

(C) Data is the most important part of a file system which also includes hardware, software people and procedures.

(D) When any of the file's data characteristics change, all data access programs are subjects to change.

7. Consider the following :

Process	Time
P	4
Q	5
Q	3
P	3
R	4

Fig. r_1

Time
3
4

Fig. r_2

Then $r_1 \div r_2$ gives

(A)

Process	Time
P	3
P	4

(B)

Process	Time
P	4
Q	3
P	3
R	4

(C)

Process
P
Q
R

(D) None of these

8. Match the following Choose the best possible option.

1.	Super Key	a.	An attribute(s) in one table whose values must either match the primary key in another table or be null.
2.	Candidate Key	b.	An attribute(s) that uniquely identifies each entity in a table
3.	Primary Key	c.	A minimal superkey
4.	Secondary Key	d.	Cannot contain null entries.
		e.	An attribute(s) used strictly for data retrieval purpose.

(A) 1 – b, 2 – d, 3 – c, 4 – a (B) 1 – d, 2 – c, 3 – b, 4 – a

(C) 1 – a, 2 – c, 3 – b, 4 – e (D) 1 – b, 2 – c, 3 – d, 4 – e

9. A natural JOIN is the result of a three–stage process. Choose the option that gives the correct order of it.

(A) PRODUCT, JOIN and PROJECT

(B) PRODUCT, JOIN and SELECT

(C) PRODUCT, PROJECT and SELECT

(D) PRODUCT, SELECT and PROJECT

Directions for Q10 – Q12

Refer table 1, table 2 and table 3 of a banking system.

Table 1

Client_name	Client_city	Client_pin
Amit	Mumbai	400086
Bhavna	Bangalore	613432
Kiran	Pune	523512
Manish	Vashi	423721
Prakash	Ratnagiri	414564
Sagar	Mumbai	400086
Vishal	Chennai	724036
Yaresh	Bangalore	614012

Fig. : The Client relation

Table 2

Acc_no	Balance
S_101	1000
S_102	1500
S_251	1900
S_256	900
S_305	1350

Fig The Account relation

Table 3

Client_name	Acc_no
Bhavna	S_101
Bhavna	S_251
Manish	S_256
Prakash	S_102
Vishal	S_305

Fig. : The Depositor relation

10. How many tuples will be present in response to the following query.

$$\pi_{(\text{Client_name, Client_pin})}(\text{Client}) \times \pi_{(\text{Client_name})}(\text{Depositor})$$

(A) 28 (B) 32

(C) 35 (D) 40

11. How many tuples and attributes will be the result of the following query
$\sigma_{Client_city="Bangalore"}(Client \bowtie Account \bowtie Depositor)$ where '$\bowtie$' is natural join.

(A) 2, 5 (B) 8, 4

(C) 8, 5 (D) 8, 6

12. Which of the following is a query to find the name of all clients who have an account in the bank and balance more than 1000.

(A) $\sigma_{(Client_name)}\left(\pi_{(Balance>100)}(Client \bowtie Account)\right)$

(B) $\pi_{(Client_name)}\left(\sigma_{(Balance>"100")}(Client \bowtie Account)\right)$

(C) $\sigma_{(Client_name)}\left(\pi_{(Balance>"100")}(Client \bowtie Account \bowtie Depositor)\right)$

(D) $\pi_{(Client_name)}\left(\sigma_{(Balance>"100")}(Client \bowtie Account \bowtie Depositor)\right)$

Directions for Q13 to Q15

Eid	Name	Age
101	Adams	41
111	Bond	48
121	Jimmy	32
131	John	53
141	Lily	45
151	Smith	39
161	Warne	59

Fig. : Manager relation

Eid	Br_id	Sal
101	NY_10	28,000
111	NY_12	32,000
121	NULL	18,000
131	AM_11	35,000
141	AM_11	32,000
151	NJ_10	28,000
161	NJ_10	38,000

Fig. : Branch relation

Br_id	Pass_word
NY_10	ABC
NY_12	PQR
AM_11	LMN
NJ_10	RST
OM_17	XYZ

Fig. : Password relation

Choose the correct option that satisfy the given query

13. Find the names of all managers whose password is XYZ

(A) $\pi_{Name}\left(\left(\sigma_{Pass_word="XYZ"}(Password)\right) \times Manager\right)$

(B) $\pi_{NAME}\left(\left(\sigma_{Pass_word="XYZ"}(Password)\right) \times Manager \times Branch\right)$

(C) $\pi_{NAME}\left(\left(\sigma_{Pass_word="XYZ"}(Password)\right) \bowtie Manager\right)$

(D) $\pi_{Name}\left(\left(\sigma_{Pass_word="XYZ"}(Password)\right) \bowtie Manager \bowtie Branch\right)$

14. Find the names of manager whose salary is more than 30,000

(A) $\pi_{\text{Name}}\left(\left(\sigma_{\text{Sal}>30000}\left(\text{Manager}\right)\right) \bowtie \text{Branch} \bowtie \text{Password}\right)$

(B) $\pi_{\text{Name}}\left(\left(\sigma_{\text{Sal}>30000}\left(\text{Manager}\right)\right) \bowtie \text{Branch}\right)$

(C) $\pi_{\text{Name}}\left(\left(\sigma_{\text{Sal}>30000}\left(\text{Branch}\right)\right) \bowtie \text{Manager}\right)$

(D) $\pi_{\text{Name}}\left(\left(\sigma_{\text{Sal}>30000}\left(\text{Branch}\right)\right) \bowtie \text{Manager} \bowtie \text{Password}\right)$

15. Find the name of managers who are sharing their password (i.e. more than one Manager at the same Branch)

(A) $\pi_{\text{Name}}\left(\sigma_{\text{Manager.Eid=Branch.Eid}}\left(\text{Manager} \bowtie \text{Branch}\right)\right)$

(B) $\pi_{\text{Name}}\left(\sigma_{\text{Manager.Br_id=Password.Br_id}}\left(\text{Manager} \bowtie \text{Branch}\right)\right)$

(C) $\pi_{\text{Name}}\left(\left(\sigma_{\text{Branch.Eid}\neq\text{x.Eid}\wedge\text{Branch.Br_id=x.Br_id}}\left(\rho_{\text{x}}\left(\text{branch}\right)\right) \bowtie \text{Password}\right) \bowtie \text{Manager}\right)$

(D) $\pi_{\text{Name}}\left(\sigma_{\text{Manager.Br_id=Manager.Br_id}}\left(\text{Manager} \bowtie \text{Branch}\right)\right)$

❑ ❑ ❑ ❑ ❑ ❑